

Advanced Machine Learning

Lecture 11: Autoencoder & GAN

Jens-Peter M. Zemke (today: Jonas Grams)
zemke@tuhh.de

Institut für Mathematik
Lehrstuhl Numerische Mathematik
Technische Universität Hamburg

14 January 2025



Outline

Repetition

- Residual neural networks & variants
- Highway Networks
- ResNet
- DenseNet

Autoencoder & GAN

- Autoencoder
- Math & fun with (fashion) MNIST
- Variational autoencoder
- GAN

Outline

Repetition

Residual neural networks & variants
Highway Networks
ResNet
DenseNet

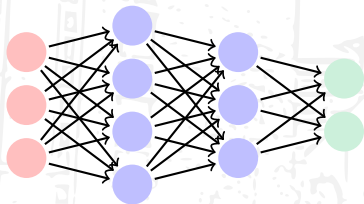
Autoencoder & GAN

Autoencoder
Math & fun with (fashion) MNIST
Variational autoencoder
GAN

Can we go deeper?

When we use one of the net structures we have and add layers, the **training takes longer**, the learning rate has to be **decreased**, and the final accuracy will **not be as good** as for less layers.

Theoretically, adding new layers at the end of an existing network **should not result in a worse approximation**, e.g., for ReLU and other variants we could always use the **identity transformation** and a **zero bias** vector to extend the net as far as we wish to.

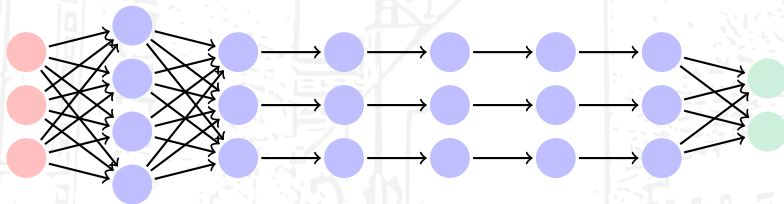


Thus, a trained net of that form **should at least be as good as** the smaller one.

Can we go deeper?

When we use one of the net structures we have and add layers, the **training takes longer**, the learning rate has to be **decreased**, and the final accuracy will **not be as good** as for less layers.

Theoretically, adding new layers at the end of an existing network **should not result in a worse approximation**, e.g., for ReLU and other variants we could always use the **identity transformation** and a **zero bias** vector to extend the net as far as we wish to.



Thus, a trained net of that form **should at least be as good as** the smaller one.

Gates in feedforward/convolutional nets

Inspired by the gates in LSTM, Srivastava, Greff, and Schmidhuber [16] **regulate the information flow** in a neural network via a learned gating mechanism. The neural network “can have paths along which information can **flow across several layers without attenuation**.” These paths are termed **Information Highways** and the resulting networks **Highway Networks**.

In **Highway Networks** we replace a layer H that maps input to output of equal dimension, e.g.,

$$\mathbf{y} = H(\mathbf{x}) = a(\mathbf{W}\mathbf{x} + \mathbf{b}), \quad \mathbf{x}, \mathbf{y} \in \mathbb{R}^n,$$

by including a **transform gate** T and a **carry gate** C (often these are coupled, $C = \mathbf{e} - T$),

$$T(\mathbf{x}) = \sigma(\mathbf{W}_t\mathbf{x} + \mathbf{b}_t), \quad C(\mathbf{x}) = \sigma(\mathbf{W}_c\mathbf{x} + \mathbf{b}_c),$$

to obtain, similar to the **cell state in an LSTM**, the mapping

$$\mathbf{y} = H(\mathbf{x}) \circ T(\mathbf{x}) + \mathbf{x} \circ C(\mathbf{x}).$$

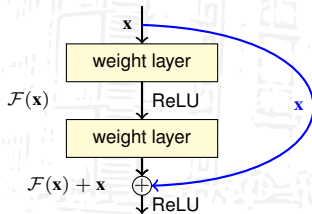
Approximating the identity

In He et al. [6] the authors pursue the idea that it is easier to **approximate a residual** than the function, e.g., for a single layer, in place of approximating $\mathcal{H} \approx \text{id}$, **they approximate $\mathcal{F}$** :

$$\mathbf{y} = \mathcal{H}(\mathbf{x}) \approx \hat{a}(\hat{\mathbf{W}}\mathbf{x} + \hat{\mathbf{b}}) \rightsquigarrow \mathcal{F}(\mathbf{x}) := \mathcal{H}(\mathbf{x}) - \mathbf{x} \approx a(\mathbf{W}\mathbf{x} + \mathbf{b}), \quad \mathbf{y} = \mathcal{F}(\mathbf{x}) + \mathbf{x}.$$

This **Residual Net** (ResNet, ResNN) is a special case of a **Highway Network** ($T = \mathbf{e}$, $C = \mathbf{e}$, $H = \mathcal{F}$).

The formulation of $\mathcal{F}(\mathbf{x}) + \mathbf{x}$ is **realized** by a feedforward neural network with **shortcut connections**:



In this example, which is used in He et al. [6],

$$\mathcal{F}(\mathbf{x}) := \mathbf{W}_2 \text{ReLU}(\mathbf{W}_1 \mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2$$

is **followed by the addition and then the activation**,

$$\mathbf{y} = \mathcal{F}(\mathbf{x}) + \mathbf{x}, \quad \mathbf{z} = \text{ReLU}(\mathbf{y}).$$

Variants of ResNet (scramble BN, ReLU, affine linear)

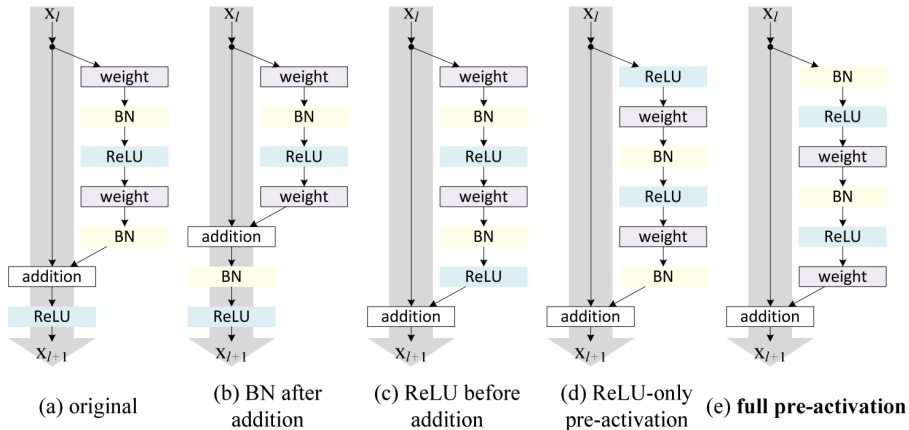
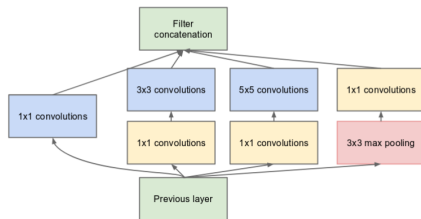


Image taken from He et al. [7].

ResNet-1001, ResNeXt

Based on the **full pre-activation variant**, He et al. [7] define a ResNet-1001 with **1001 layers** that they successfully trained.

ResNeXt: Xie et al. [19] combine **ResNet** with the **Inception modules** of GoogLeNet V1 from Szegedy et al. [18],



(b) Inception module with dimension reductions

Image taken from Szegedy et al. [18].

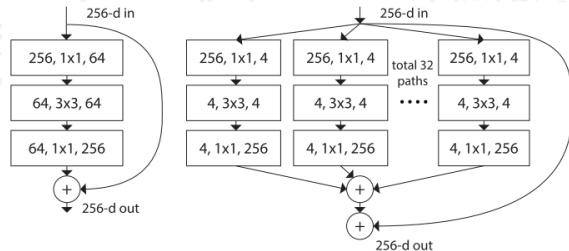
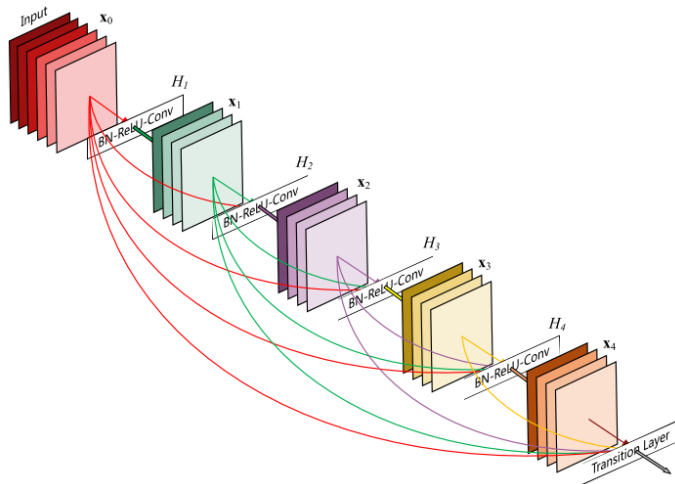


Image taken from Xie et al. [19].

Adding more paths



Huang et al. [9] use **every feature map** ever constructed in every later layer. The resulting net is called a **DenseNet**.

A DenseNet has

$$\frac{L(L+1)}{2}$$

shortcut connections.

In contrast to a **ResNet**, the inputs are **concatenated**.

Image taken from Huang et al. [9].

Outline

Repetition

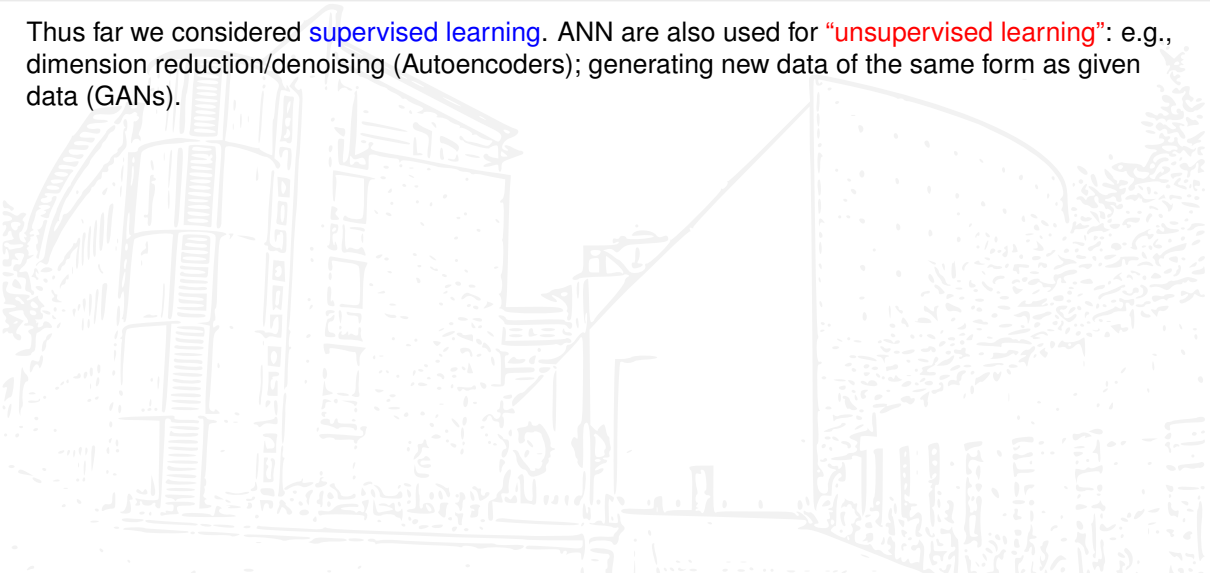
- Residual neural networks & variants
- Highway Networks
- ResNet
- DenseNet

Autoencoder & GAN

- Autoencoder
- Math & fun with (fashion) MNIST
- Variational autoencoder
- GAN

Motivation

Thus far we considered **supervised learning**. ANN are also used for “**unsupervised learning**”: e.g., dimension reduction/denoising (Autoencoders); generating new data of the same form as given data (GANs).



Motivation

Thus far we considered **supervised learning**. ANN are also used for “**unsupervised learning**”: e.g., dimension reduction/denoising (Autoencoders); generating new data of the same form as given data (GANs).

- ▶ **Autoencoders** are neural networks trained on given (perturbed) data that try to compute the (unperturbed) data, i.e., they (mostly) implement a mapping $\mathbf{f}_{\text{autoencoder}} : \mathbb{R}^n \rightarrow \mathbb{R}^n$. Most autoencoders feature a layer w/ lower dimension $m \ll n$ that acts as a bottleneck, the part up to that layer is known as **encoder** $\mathbf{f}_{\text{encoder}} : \mathbb{R}^n \rightarrow \mathbb{R}^m$, the part from that layer on as **decoder** $\mathbf{f}_{\text{decoder}} : \mathbb{R}^m \rightarrow \mathbb{R}^n$, $\mathbf{f}_{\text{autoencoder}} = \mathbf{f}_{\text{decoder}} \circ \mathbf{f}_{\text{encoder}}$.

Motivation

Thus far we considered **supervised learning**. ANN are also used for “**unsupervised learning**”: e.g., dimension reduction/denoising (Autoencoders); generating new data of the same form as given data (GANs).

- ▶ **Autoencoders** are neural networks trained on given (perturbed) data that try to compute the (unperturbed) data, i.e., they (mostly) implement a mapping $\mathbf{f}_{\text{autoencoder}} : \mathbb{R}^n \rightarrow \mathbb{R}^n$. Most autoencoders feature a layer w/ lower dimension $m \ll n$ that acts as a bottleneck, the part up to that layer is known as **encoder** $\mathbf{f}_{\text{encoder}} : \mathbb{R}^n \rightarrow \mathbb{R}^m$, the part from that layer on as **decoder** $\mathbf{f}_{\text{decoder}} : \mathbb{R}^m \rightarrow \mathbb{R}^n$, $\mathbf{f}_{\text{autoencoder}} = \mathbf{f}_{\text{decoder}} \circ \mathbf{f}_{\text{encoder}}$.
- ▶ A **generative adversarial network (GAN)** is a neural network consisting of competitive parts with contrastive aims. In the simplest form we have two networks, a **generator** and a **discriminator** network: the generator generates data that should be similar to given data, the discriminator distinguishes between original and generated data. Both networks compete with each other during training.

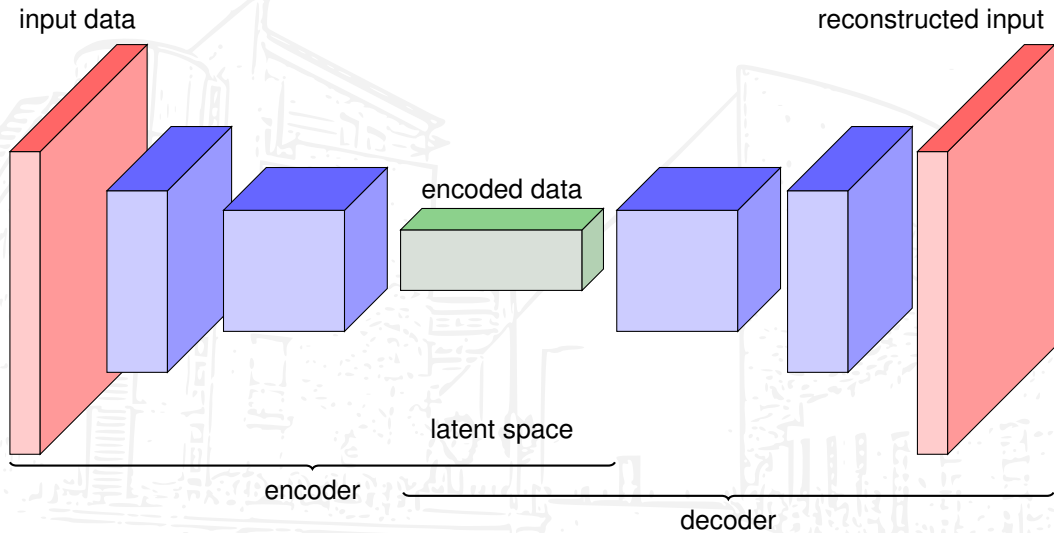
Motivation

Thus far we considered supervised learning. ANN are also used for “unsupervised learning”: e.g., dimension reduction/denoising (Autoencoders); generating new data of the same form as given data (GANs).

- ▶ **Autoencoders** are neural networks trained on given (perturbed) data that try to compute the (unperturbed) data, i.e., they (mostly) implement a mapping $\mathbf{f}_{\text{autoencoder}} : \mathbb{R}^n \rightarrow \mathbb{R}^n$. Most autoencoders feature a layer w/ lower dimension $m \ll n$ that acts as a bottleneck, the part up to that layer is known as **encoder** $\mathbf{f}_{\text{encoder}} : \mathbb{R}^n \rightarrow \mathbb{R}^m$, the part from that layer on as **decoder** $\mathbf{f}_{\text{decoder}} : \mathbb{R}^m \rightarrow \mathbb{R}^n$, $\mathbf{f}_{\text{autoencoder}} = \mathbf{f}_{\text{decoder}} \circ \mathbf{f}_{\text{encoder}}$.
- ▶ A **generative adversarial network (GAN)** is a neural network consisting of competitive parts with contrastive aims. In the simplest form we have two networks, a **generator** and a **discriminator** network: the generator generates data that should be similar to given data, the discriminator distinguishes between original and generated data. Both networks compete with each other during training.

Both ideas share common properties, we start with various forms of autoencoders.

Autoencoder: pictorial definition



Autoencoder: mathematical formulation & implementation

We need a **network structure**:

- ▶ **Encoder**: using dense layers we can use any of the networks for classification and replace the last layer by a linear one.
- ▶ **Decoder**: use a reversed network, often the same structure as that of the reversed encoder.
- ▶ **CNN-Autoencoder**: we need **upsampling** and **transposed convolutions**.

Autoencoder: mathematical formulation & implementation

We need a **network structure**:

- ▶ **Encoder**: using dense layers we can use any of the networks for classification and replace the last layer by a linear one.
- ▶ **Decoder**: use a reversed network, often the same structure as that of the reversed encoder.
- ▶ **CNN-Autoencoder**: we need **upsampling** and **transposed convolutions**.

We need a **loss function**, candidates are:

- ▶ **mean squared error (MSE)** for continuous data $\mathbf{x}, \tilde{\mathbf{x}} \in \mathbb{R}^n$:

$$MSE(\mathbf{x}, \tilde{\mathbf{x}}) := \frac{1}{n} \|\mathbf{x} - \tilde{\mathbf{x}}\|_2^2, \quad \tilde{\mathbf{x}} := \mathbf{f}_{\text{autoencoder}}(\mathbf{x})$$

- ▶ **(binary) cross-entropy** for probability data $\mathbf{x}, \tilde{\mathbf{x}} \in [0, 1]^n$ (or binary data $\mathbf{x} \in \{0, 1\}^n$):

$$H(\mathbf{x}, \tilde{\mathbf{x}}) := \sum_{i=1}^n -x_i \log(\tilde{x}_i), \quad \tilde{\mathbf{x}} := \mathbf{f}_{\text{autoencoder}}(\mathbf{x})$$

Autoencoder: applications

Why do autoencoders work? $\rightsquigarrow$ manifold hypothesis



Autoencoder: applications

Why do autoencoders work? $\rightsquigarrow$ manifold hypothesis

Applications of autoencoders:

- ▶ dimension reduction (sometimes as preliminary step for, e.g., SNE/t-SNE/UMAP)
- ▶ denoising: perturb input during training by noise, e.g., Gaussian blur, salt-and-pepper, corruption, obfuscation, try to obtain unperturbed data
- ▶ coloring of images: special perturbation = grayscale image as input
- ▶ images from cartoons/sketches
- ▶ compression (encode, transmit, decode)
- ▶ ...

Autoencoder: selected applications

Dimension reduction:

- ▶ principal component analysis (PCA) = singular value decomposition (SVD): linear, cheap
- ▶ stochastic neighbor embedding (SNE) [8]: nonlinear, costly
- ▶ t-distributed stochastic neighbor embedding (t-SNE) [12]: nonlinear, costly
- ▶ uniform manifold approximation and projection (UMAP) [13]: nonlinear, costly
- ▶ **Autoencoder**: linear or nonlinear, from cheap to costly, can be combined w/ the above

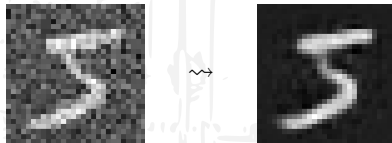
Autoencoder: selected applications

Dimension reduction:

- ▶ principal component analysis (PCA) = singular value decomposition (SVD): linear, cheap
- ▶ stochastic neighbor embedding (SNE) [8]: nonlinear, costly
- ▶ t-distributed stochastic neighbor embedding (t-SNE) [12]: nonlinear, costly
- ▶ uniform manifold approximation and projection (UMAP) [13]: nonlinear, costly
- ▶ **Autoencoder**: linear or nonlinear, from cheap to costly, can be combined w/ the above

Application example:

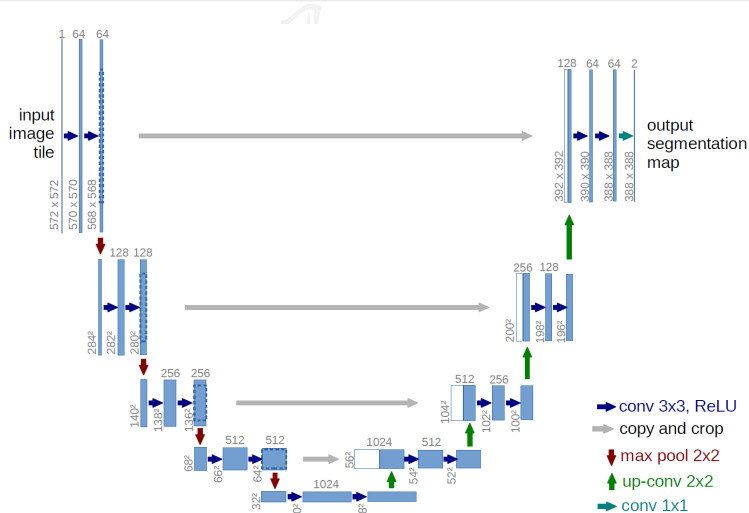
- ▶ denoising, e.g., train on MNIST, perturb input by noise:



Pushing it further: U-Net

Q: Can we combine autoencoders w/ skip connections + convolutions?

Pushing it further: U-Net

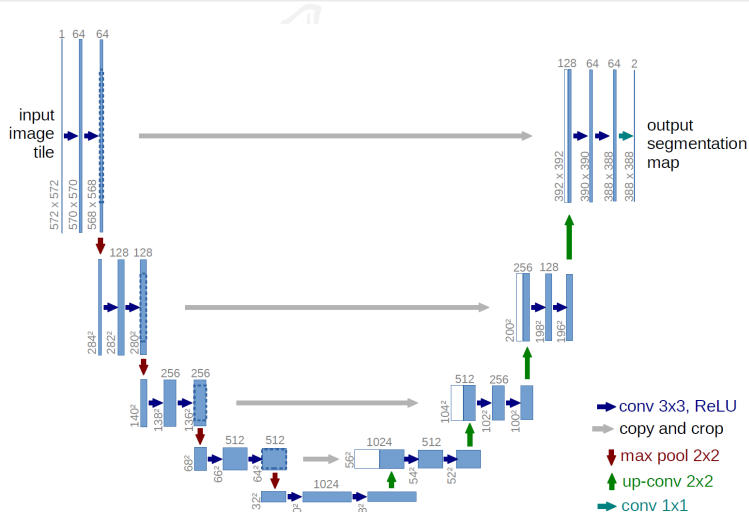


Q: Can we combine autoencoders w/ skip connections + convolutions?

A: Yes, an example is **U-Net** by Ronneberger, Fischer, and Brox [15].

Figure taken from <https://lmb.informatik.uni-freiburg.de/people/ronneber/u-net/>

Pushing it further: U-Net



Q: Can we combine autoencoders w/ skip connections + convolutions?

A: Yes, an example is **U-Net** by Ronneberger, Fischer, and Brox [15].

U-Net is the **de-facto-standard** for **biomedical image segmentation**

(To be fair: grouping U-Net as Autoencoder pushes borders a bit far)

Figure taken from <https://lmb.informatik.uni-freiburg.de/people/ronneber/u-net/>

Linear autoencoder = PCA = SVD

A **simple autoencoder** (linear, one layer each) corresponds to **principal component analysis** (PCA), i.e., approximation using the **SVD** $\mathbf{X} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$ of the data, see Bourlard and Kamp [3]:

$$\min_{\mathbf{W}_1, \mathbf{W}_2} \|\mathbf{X} - \mathbf{W}_2 \mathbf{W}_1 \mathbf{X}\|_F = \|\mathbf{X} - \mathbf{V}_k \mathbf{U}_k^T \mathbf{X}\|_F, \quad \mathbf{V}_k = (\mathbf{v}_1 \quad \cdots \quad \mathbf{v}_k), \quad \mathbf{U}_k = (\mathbf{u}_1 \quad \cdots \quad \mathbf{u}_k).$$

Linear autoencoder = PCA = SVD

A **simple autoencoder** (linear, one layer each) corresponds to **principal component analysis** (PCA), i.e., approximation using the **SVD** $\mathbf{X} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$ of the data, see Bourlard and Kamp [3]:

$$\min_{\mathbf{W}_1, \mathbf{W}_2} \|\mathbf{X} - \mathbf{W}_2 \mathbf{W}_1 \mathbf{X}\|_F = \|\mathbf{X} - \mathbf{V}_k \mathbf{U}_k^T \mathbf{X}\|_F, \quad \mathbf{V}_k = (\mathbf{v}_1 \quad \cdots \quad \mathbf{v}_k), \quad \mathbf{U}_k = (\mathbf{u}_1 \quad \cdots \quad \mathbf{u}_k).$$

This follows from the so-called **Schmidt-Mirsky-Theorem** (Stewart and Sun [17, Theorem 4.18]):

Theorem (Schmidt, Mirsky)

Let $\mathbf{X} \in \mathbb{R}^{m \times n}$, $k \leq \min(m, n)$. Then for the Frobenius and 2-norms

$$\|\mathbf{X} - \mathbf{X}_k\| = \min_{\text{rank}(\mathbf{A})=k} \|\mathbf{X} - \mathbf{A}\|, \quad \mathbf{X}_k = \mathbf{U}_k \mathbf{\Sigma}_k \mathbf{V}_k^T, \quad \mathbf{\Sigma}_k := \text{diag}(\sigma_1, \dots, \sigma_k).$$

Linear autoencoder = PCA = SVD

A **simple autoencoder** (linear, one layer each) corresponds to **principal component analysis** (PCA), i.e., approximation using the **SVD** $\mathbf{X} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$ of the data, see Bourlard and Kamp [3]:

$$\min_{\mathbf{W}_1, \mathbf{W}_2} \|\mathbf{X} - \mathbf{W}_2 \mathbf{W}_1 \mathbf{X}\|_F = \|\mathbf{X} - \mathbf{V}_k \mathbf{U}_k^T \mathbf{X}\|_F, \quad \mathbf{V}_k = (\mathbf{v}_1 \quad \cdots \quad \mathbf{v}_k), \quad \mathbf{U}_k = (\mathbf{u}_1 \quad \cdots \quad \mathbf{u}_k).$$

This follows from the so-called **Schmidt-Mirsky-Theorem** (Stewart and Sun [17, Theorem 4.18]):

Theorem (Schmidt, Mirsky)

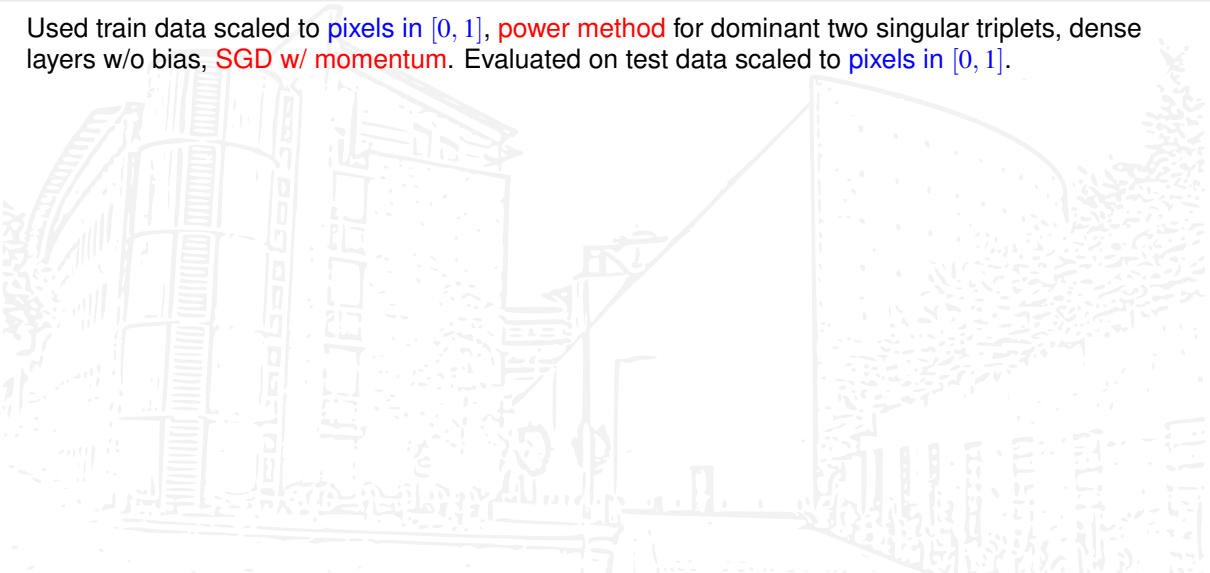
Let $\mathbf{X} \in \mathbb{R}^{m \times n}$, $k \leq \min(m, n)$. Then for the Frobenius and 2-norms

$$\|\mathbf{X} - \mathbf{X}_k\| = \min_{\text{rank}(\mathbf{A})=k} \|\mathbf{X} - \mathbf{A}\|, \quad \mathbf{X}_k = \mathbf{U}_k \mathbf{\Sigma}_k \mathbf{V}_k^T, \quad \mathbf{\Sigma}_k := \text{diag}(\sigma_1, \dots, \sigma_k).$$

We compute the first **two singular triplets** of **Fashion MNIST** and compare with the latent space of a trained **linear autoencoder**.

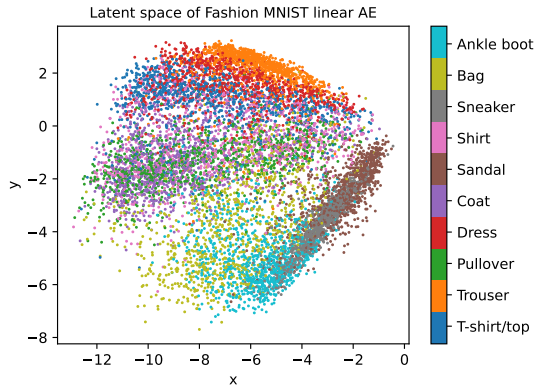
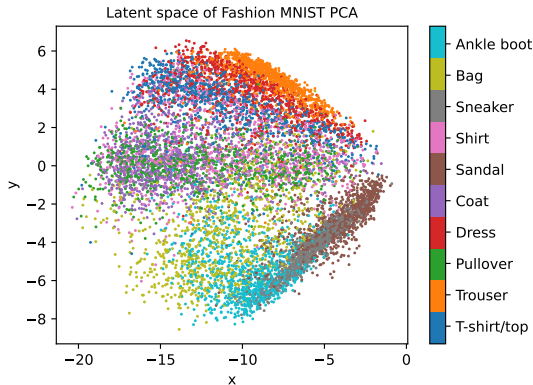
Fashion MNIST: PCA vs. linear AE

Used train data scaled to **pixels in $[0, 1]$** , **power method** for dominant two singular triplets, dense layers w/o bias, **SGD w/ momentum**. Evaluated on test data scaled to **pixels in $[0, 1]$** .



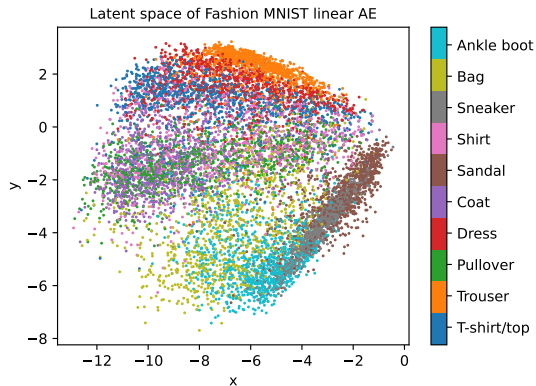
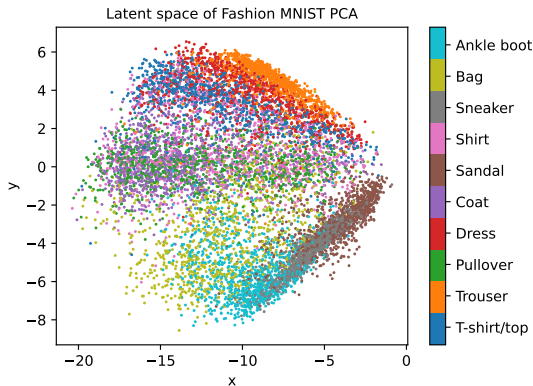
Fashion MNIST: PCA vs. linear AE

Used train data scaled to pixels in $[0, 1]$, **power method** for dominant two singular triplets, dense layers w/o bias, **SGD w/ momentum**. Evaluated on test data scaled to pixels in $[0, 1]$.



Fashion MNIST: PCA vs. linear AE

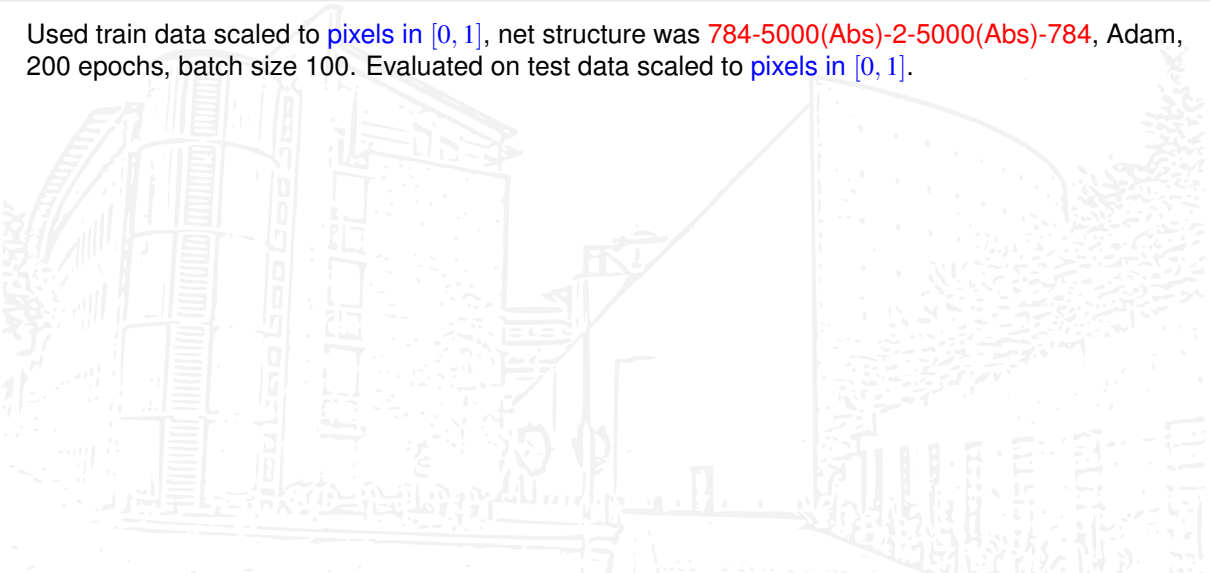
Used train data scaled to pixels in $[0, 1]$, **power method** for dominant two singular triplets, dense layers w/o bias, **SGD w/ momentum**. Evaluated on test data scaled to pixels in $[0, 1]$.



We only know $\mathbf{W}_2\mathbf{W}_1$ converges to $\mathbf{V}_k\mathbf{U}_k^T$, but this implies that $\mathbf{W}_1$ and $\mathbf{U}_k^T$ span the same space.

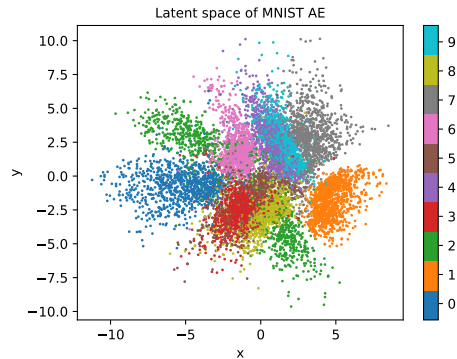
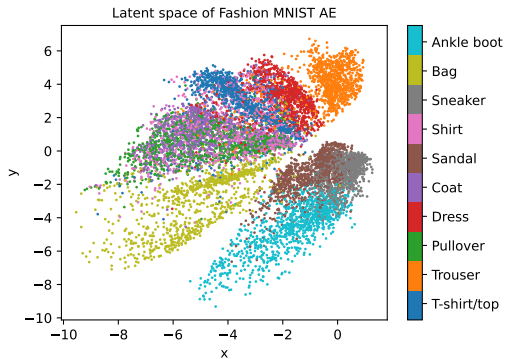
Genuine AE: Fashion MNIST & MNIST

Used train data scaled to **pixels in $[0, 1]$** , net structure was **784-5000(Abs)-2-5000(Abs)-784**, Adam, 200 epochs, batch size 100. Evaluated on test data scaled to **pixels in $[0, 1]$** .



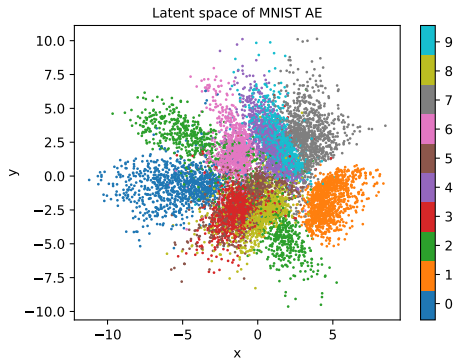
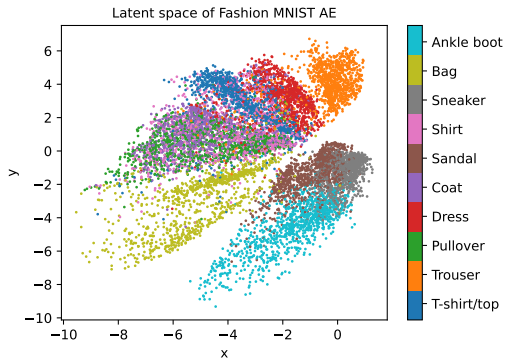
Genuine AE: Fashion MNIST & MNIST

Used train data scaled to pixels in $[0, 1]$, net structure was 784-5000(Abs)-2-5000(Abs)-784, Adam, 200 epochs, batch size 100. Evaluated on test data scaled to pixels in $[0, 1]$.



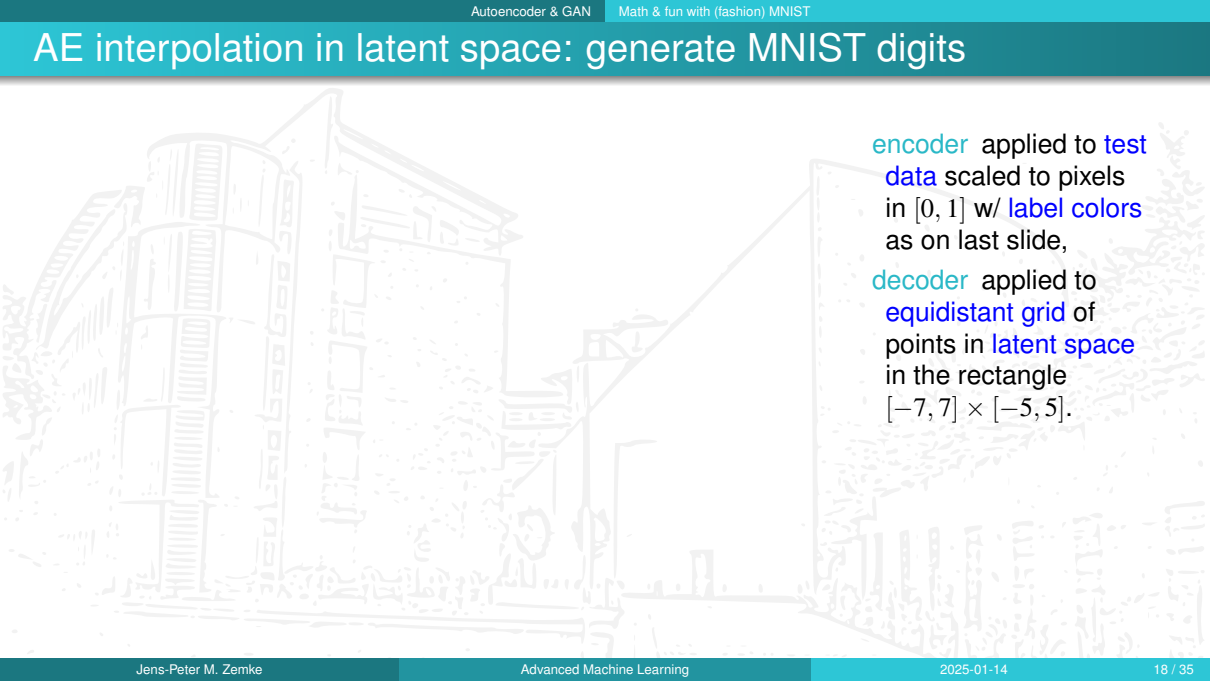
Genuine AE: Fashion MNIST & MNIST

Used train data scaled to pixels in $[0, 1]$, net structure was 784-5000(Abs)-2-5000(Abs)-784, Adam, 200 epochs, batch size 100. Evaluated on test data scaled to pixels in $[0, 1]$.



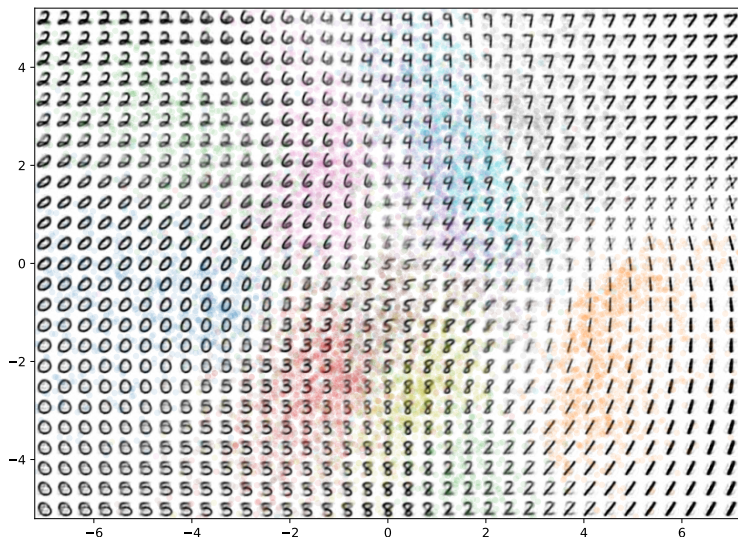
Reminder: **the net does not know the labels!** These clusters arise as **best two-dimensional representation** of the data in latent space to **enable reconstruction** of the input on output.

AE interpolation in latent space: generate MNIST digits



encoder applied to test data scaled to pixels in $[0, 1]$ w/ label colors as on last slide,
decoder applied to equidistant grid of points in latent space in the rectangle $[-7, 7] \times [-5, 5]$.

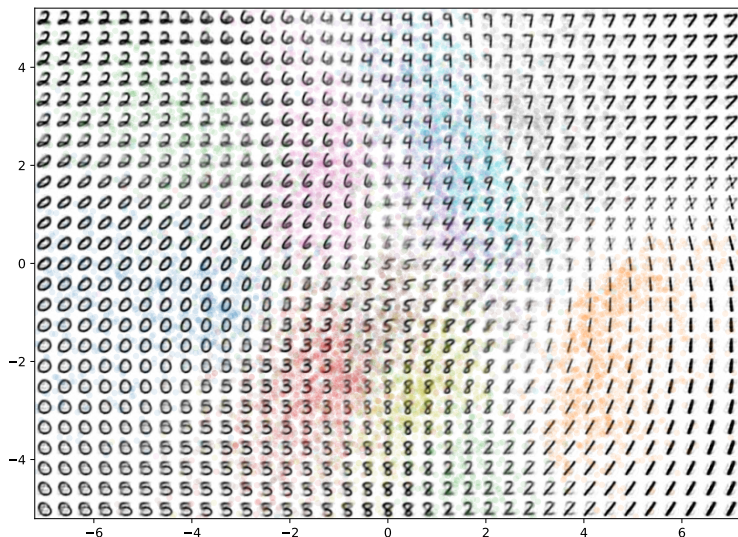
AE interpolation in latent space: generate MNIST digits



encoder applied to test data scaled to pixels in $[0, 1]$ w/ label colors as on last slide,

decoder applied to equidistant grid of points in latent space in the rectangle $[-7, 7] \times [-5, 5]$.

AE interpolation in latent space: generate MNIST digits



encoder applied to test data scaled to pixels in $[0, 1]$ w/ label colors as on last slide,

decoder applied to equidistant grid of points in latent space in the rectangle $[-7, 7] \times [-5, 5]$.

Observations:

- ▶ in white regions funny “digits”
- ▶ digits somewhat blurry

Information geometry: Bregman divergence

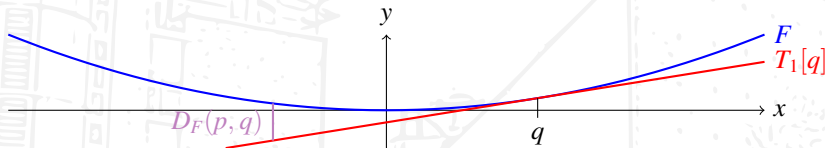
Let $F \in C^1$ be **strictly convex**, then **Bregman divergence** defined by

$$D_F(\mathbf{p}, \mathbf{q}) := F(\mathbf{p}) - F(\mathbf{q}) - \langle \nabla F(\mathbf{q}), \mathbf{p} - \mathbf{q} \rangle = F(\mathbf{p}) - T_1[\mathbf{q}](\mathbf{p}).$$

Information geometry: Bregman divergence

Let $F \in C^1$ be strictly convex, then Bregman divergence defined by

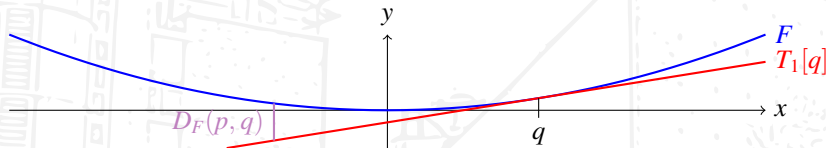
$$D_F(\mathbf{p}, \mathbf{q}) := F(\mathbf{p}) - F(\mathbf{q}) - \langle \nabla F(\mathbf{q}), \mathbf{p} - \mathbf{q} \rangle = F(\mathbf{p}) - T_1[\mathbf{q}](\mathbf{p}).$$



Information geometry: Bregman divergence

Let $F \in C^1$ be **strictly convex**, then **Bregman divergence** defined by

$$D_F(\mathbf{p}, \mathbf{q}) := F(\mathbf{p}) - F(\mathbf{q}) - \langle \nabla F(\mathbf{q}), \mathbf{p} - \mathbf{q} \rangle = F(\mathbf{p}) - T_1[\mathbf{q}](\mathbf{p}).$$



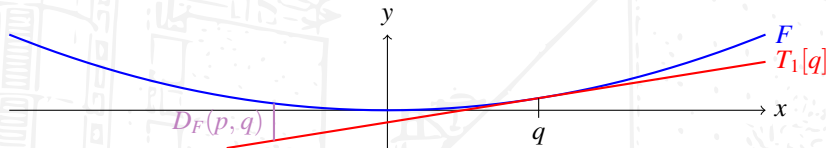
Properties:

- ▶ **non-negative**: $D_F(\mathbf{p}, \mathbf{q}) \geq 0$ for all $\mathbf{p}, \mathbf{q}$, only zero for $\mathbf{p} = \mathbf{q}$
- ▶ **convex in first argument** (not necessarily in second)
- ▶ in general **not symmetric**, i.e., $D_F(\mathbf{p}, \mathbf{q}) \neq D_F(\mathbf{q}, \mathbf{p})$ possible
- ▶ **mean minimizes** expected Bregman divergence

Information geometry: Bregman divergence

Let $F \in C^1$ be **strictly convex**, then **Bregman divergence** defined by

$$D_F(\mathbf{p}, \mathbf{q}) := F(\mathbf{p}) - F(\mathbf{q}) - \langle \nabla F(\mathbf{q}), \mathbf{p} - \mathbf{q} \rangle = F(\mathbf{p}) - T_1[\mathbf{q}](\mathbf{p}).$$



Properties:

- ▶ **non-negative**: $D_F(\mathbf{p}, \mathbf{q}) \geq 0$ for all $\mathbf{p}, \mathbf{q}$, only zero for $\mathbf{p} = \mathbf{q}$
- ▶ **convex in first argument** (not necessarily in second)
- ▶ in general **not symmetric**, i.e., $D_F(\mathbf{p}, \mathbf{q}) \neq D_F(\mathbf{q}, \mathbf{p})$ possible
- ▶ **mean minimizes** expected Bregman divergence

Classical example: let $F(\mathbf{p}) = \|\mathbf{p}\|^2$, then $D_F(\mathbf{p}, \mathbf{q}) = \|\mathbf{p} - \mathbf{q}\|^2$ is **squared distance**

Information geometry: Kullback-Leibler divergence

Let $\mathbf{p}, \mathbf{q}$ be **discrete probability vectors**, i.e.,

$$0 \leq p_i, q_i \leq 1, \quad i = 1, \dots, n, \quad \sum_{i=1}^n p_i = \sum_{i=1}^n q_i = 1.$$

Information geometry: Kullback-Leibler divergence

Let $\mathbf{p}, \mathbf{q}$ be **discrete probability vectors**, i.e.,

$$0 \leq p_i, q_i \leq 1, \quad i = 1, \dots, n, \quad \sum_{i=1}^n p_i = \sum_{i=1}^n q_i = 1.$$

Then the **Bregman divergence** of the **negative entropy** is the **Kullback-Leibler divergence**,

$$F(\mathbf{p}) := -H(\mathbf{p}) := \sum_{i=1}^n p_i \log(p_i) \quad \Rightarrow \quad D_{\text{KL}}(\mathbf{p} \parallel \mathbf{q}) := D_F(\mathbf{p}, \mathbf{q}) = \sum_{i=1}^n p_i \log \left(\frac{p_i}{q_i} \right) = \mathbb{E}_P \left[\log \frac{P}{Q} \right].$$

Information geometry: Kullback-Leibler divergence

Let $\mathbf{p}, \mathbf{q}$ be **discrete probability vectors**, i.e.,

$$0 \leq p_i, q_i \leq 1, \quad i = 1, \dots, n, \quad \sum_{i=1}^n p_i = \sum_{i=1}^n q_i = 1.$$

Then the **Bregman divergence** of the **negative entropy** is the **Kullback-Leibler divergence**,

$$F(\mathbf{p}) := -H(\mathbf{p}) := \sum_{i=1}^n p_i \log(p_i) \quad \Rightarrow \quad D_{\text{KL}}(\mathbf{p} \parallel \mathbf{q}) := D_F(\mathbf{p}, \mathbf{q}) = \sum_{i=1}^n p_i \log\left(\frac{p_i}{q_i}\right) = \mathbb{E}_P \left[\log \frac{P}{Q} \right].$$

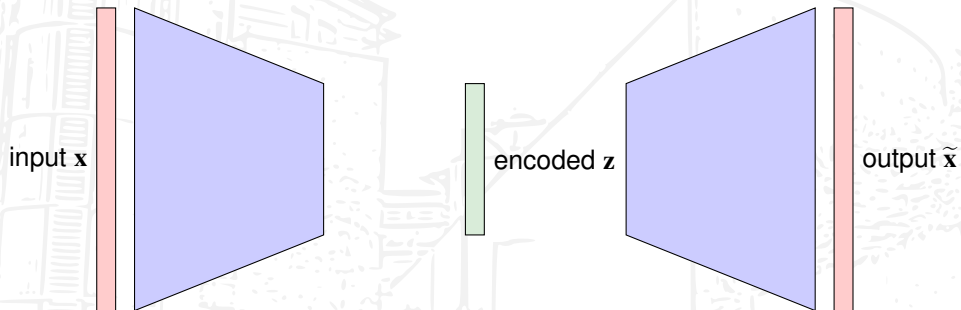
Kullback-Leibler for **normal distributions** & **simple subcase** ($\Sigma = \text{diag}(\sigma_1^2, \dots, \sigma_n^2)$ diagonal):

$$D_{\text{KL}}(\mathcal{N}_0 \parallel \mathcal{N}_1) = \frac{1}{2} \left(\text{trace}(\Sigma_1^{-1} \Sigma_0) + (\boldsymbol{\mu}_1 - \boldsymbol{\mu}_0)^T \Sigma_1^{-1} (\boldsymbol{\mu}_1 - \boldsymbol{\mu}_0) - n + \ln \left(\frac{\det \Sigma_1}{\det \Sigma_0} \right) \right),$$

$$D_{\text{KL}}(\mathcal{N}(\boldsymbol{\mu}, \Sigma) \parallel \mathcal{N}(\mathbf{o}, \mathbf{I})) = \frac{1}{2} \sum_{i=1}^n (\sigma_i^2 + \mu_i^2 - 1 - \ln(\sigma_i^2)).$$

Variational autoencoder (VAE)

- In an **autoencoder** the latent space is modelled by, e.g., vectors $\mathbf{z}$

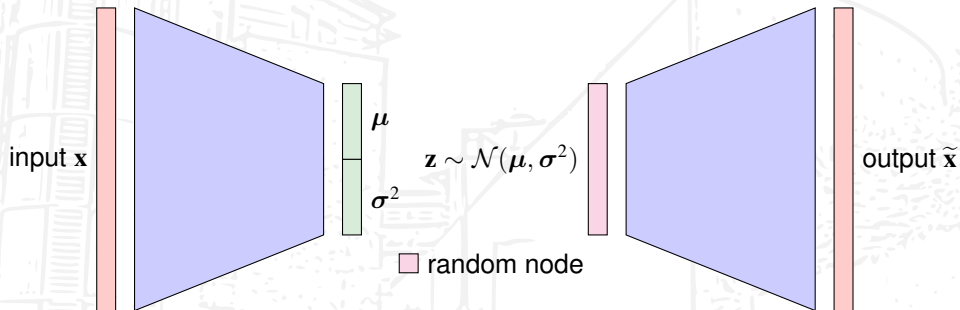


Loss function of an autoencoder:

$$C_{\text{reconstruction}}(\mathbf{x})$$

Variational autoencoder (VAE)

- ▶ In an **autoencoder** the latent space is modelled by, e.g., vectors $\mathbf{z}$
- ▶ In a **variational autoencoder** the latent space is modelled by a (diagonal normal) distribution

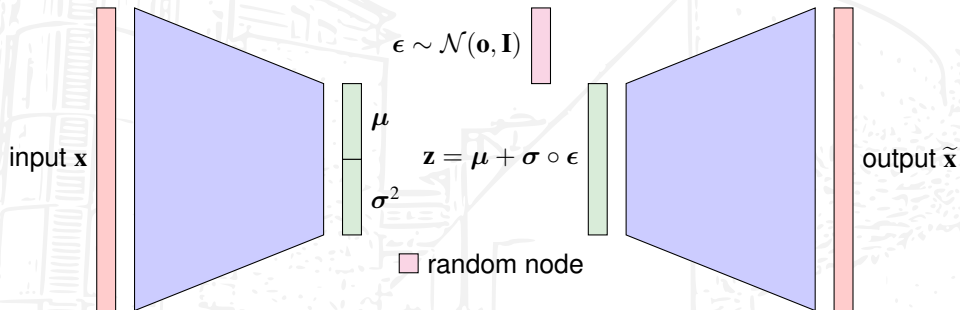


Loss function of a variational autoencoder:

$$C_{\text{vae}}(\mathbf{x}) = \text{const} \cdot C_{\text{reconstruction}}(\mathbf{x}) + C_{\text{Kullback-Leibler}}(\mathcal{N}(\mu, \sigma^2), \mathcal{N}(\mathbf{o}, \mathbf{I}))$$

Variational autoencoder (VAE)

- ▶ In an **autoencoder** the latent space is modelled by, e.g., vectors $\mathbf{z}$
- ▶ In a **variational autoencoder** the latent space is modelled by a (diagonal normal) distribution
- ▶ The “**reparameterization trick**” enables classical backpropagation



Loss function of a variational autoencoder:

$$C_{\text{vae}}(\mathbf{x}) = \text{const} \cdot C_{\text{reconstruction}}(\mathbf{x}) + C_{\text{Kullback-Leibler}}(\mathcal{N}(\mu, \sigma^2), \mathcal{N}(\mathbf{o}, \mathbf{I}))$$

Implementation of a variational autoencoder

How to implement a variational autoencoder (VAE)?

```

1  class VAE(keras.Model):
2      def __init__(self, encoder, decoder, **kwargs):
3          super(VAE, self).__init__(**kwargs)
4          self.encoder = encoder
5          self.decoder = decoder
6          self.total_loss_tracker = keras.metrics.Mean(name="total_loss")
7          self.reconstruction_loss_tracker = keras.metrics.Mean(
8              name="reconstruction_loss"
9          )
10         self.kl_loss_tracker = keras.metrics.Mean(name="kl_loss")
11
12     @property
13     def metrics(self):
14         return [
15             self.total_loss_tracker,
16             self.reconstruction_loss_tracker,
17             self.kl_loss_tracker,
18         ]
19
20     def train_step(self, data):
21         with tf.GradientTape() as tape:
22             z_mean, z_log_var, z = self.encoder(data)
23             reconstruction = self.decoder(z)
24             reconstruction_loss = tf.reduce_mean(
25                 keras.losses.binary_crossentropy(data, reconstruction)
26             )
27             kl_loss = -0.5 * (1 + z_log_var - tf.square(z_mean) - tf.exp(z_log_var))
28             kl_loss = tf.reduce_mean(tf.reduce_sum(kl_loss, axis=1))
29             total_loss = reconstruction_loss + kl_loss
30             grads = tape.gradient(total_loss, self.trainable_weights)
31             self.optimizer.apply_gradients(zip(grads, self.trainable_weights))
32             self.total_loss_tracker.update_state(total_loss)
33             self.reconstruction_loss_tracker.update_state(reconstruction_loss)
34             self.kl_loss_tracker.update_state(kl_loss)
35         return {
36             "loss": self.total_loss_tracker.result(),
37             "reconstruction_loss": self.reconstruction_loss_tracker.result(),
38             "kl_loss": self.kl_loss_tracker.result(),
39         }

```

Implementation of a variational autoencoder

How to implement a **variational autoencoder** (VAE)?

- Use **backpropagation through probabilistic layers**, e.g., the **TensorFlow Probability** package.

```

1 class VAE(keras.Model):
2     def __init__(self, encoder, decoder, **kwargs):
3         super(VAE, self).__init__(**kwargs)
4         self.encoder = encoder
5         self.decoder = decoder
6         self.total_loss_tracker = keras.metrics.Mean(name="total_loss")
7         self.reconstruction_loss_tracker = keras.metrics.Mean(
8             name="reconstruction_loss"
9         )
10        self.kl_loss_tracker = keras.metrics.Mean(name="kl_loss")
11
12    @property
13    def metrics(self):
14        return [
15            self.total_loss_tracker,
16            self.reconstruction_loss_tracker,
17            self.kl_loss_tracker,
18        ]
19
20    def train_step(self, data):
21        with tf.GradientTape() as tape:
22            z_mean, z_log_var, z = self.encoder(data)
23            reconstruction = self.decoder(z)
24            reconstruction_loss = tf.reduce_mean(
25                keras.losses.binary_crossentropy(data, reconstruction)
26            )
27            kl_loss = -0.5 * (1 + z_log_var - tf.square(z_mean) - tf.exp(z_log_var))
28            kl_loss = tf.reduce_mean(tf.reduce_sum(kl_loss, axis=1))
29            total_loss = reconstruction_loss + kl_loss
30            grads = tape.gradient(total_loss, self.trainable_weights)
31            self.optimizer.apply_gradients(zip(grads, self.trainable_weights))
32            self.total_loss_tracker.update_state(total_loss)
33            self.reconstruction_loss_tracker.update_state(reconstruction_loss)
34            self.kl_loss_tracker.update_state(kl_loss)
35        return {
36            "loss": self.total_loss_tracker.result(),
37            "reconstruction_loss": self.reconstruction_loss_tracker.result(),
38            "kl_loss": self.kl_loss_tracker.result(),
39        }

```

Implementation of a variational autoencoder

How to implement a variational autoencoder (VAE)?

- Use backpropagation through probabilistic layers, e.g., the TensorFlow Probability package.
- Write adapted training routines. Snippet on the right is from a Keras example for a VAE. Observe that we generate the log of the variance.

```

1 class VAE(keras.Model):
2     def __init__(self, encoder, decoder, **kwargs):
3         super(VAE, self).__init__(**kwargs)
4         self.encoder = encoder
5         self.decoder = decoder
6         self.total_loss_tracker = keras.metrics.Mean(name="total_loss")
7         self.reconstruction_loss_tracker = keras.metrics.Mean(
8             name="reconstruction_loss"
9         )
10        self.kl_loss_tracker = keras.metrics.Mean(name="kl_loss")
11
12    @property
13    def metrics(self):
14        return [
15            self.total_loss_tracker,
16            self.reconstruction_loss_tracker,
17            self.kl_loss_tracker,
18        ]
19
20    def train_step(self, data):
21        with tf.GradientTape() as tape:
22            z_mean, z_log_var, z = self.encoder(data)
23            reconstruction = self.decoder(z)
24            reconstruction_loss = tf.reduce_mean(
25                keras.losses.binary_crossentropy(data, reconstruction)
26            )
27            kl_loss = -0.5 * (1 + z_log_var - tf.square(z_mean) - tf.exp(z_log_var))
28            kl_loss = tf.reduce_mean(tf.reduce_sum(kl_loss, axis=1))
29            total_loss = reconstruction_loss + kl_loss
30            grads = tape.gradient(total_loss, self.trainable_weights)
31            self.optimizer.apply_gradients(zip(grads, self.trainable_weights))
32            self.total_loss_tracker.update_state(total_loss)
33            self.reconstruction_loss_tracker.update_state(reconstruction_loss)
34            self.kl_loss_tracker.update_state(kl_loss)
35        return {
36            "loss": self.total_loss_tracker.result(),
37            "reconstruction_loss": self.reconstruction_loss_tracker.result(),
38            "kl_loss": self.kl_loss_tracker.result(),
39        }

```

VAE: Bayesian interpretation

Bayes' rule: suppose $\mathbf{z}$, $\mathbf{x}$ are events, p denotes probability, and $p(\mathbf{x}) \neq 0$. Then

$$p(\mathbf{z} \mid \mathbf{x}) = \frac{p(\mathbf{x} \mid \mathbf{z})p(\mathbf{z})}{p(\mathbf{x})} \quad \Rightarrow \quad p(\mathbf{z} \mid \mathbf{x}) \propto p(\mathbf{x} \mid \mathbf{z})p(\mathbf{z}),$$

$p(\mathbf{z} \mid \mathbf{x})$ **posterior probability**, $p(\mathbf{x} \mid \mathbf{z})$ **likelihood** of $\mathbf{z}$ given $\mathbf{x}$, $p(\mathbf{z})$ **prior probability**, $p(\mathbf{x})$ **evidence**.

VAE: Bayesian interpretation

Bayes' rule: suppose $\mathbf{z}, \mathbf{x}$ are events, p denotes probability, and $p(\mathbf{x}) \neq 0$. Then

$$p(\mathbf{z} | \mathbf{x}) = \frac{p(\mathbf{x} | \mathbf{z})p(\mathbf{z})}{p(\mathbf{x})} \Rightarrow p(\mathbf{z} | \mathbf{x}) \propto p(\mathbf{x} | \mathbf{z})p(\mathbf{z}),$$

$p(\mathbf{z} | \mathbf{x})$ **posterior probability**, $p(\mathbf{x} | \mathbf{z})$ **likelihood** of $\mathbf{z}$ given $\mathbf{x}$, $p(\mathbf{z})$ **prior probability**, $p(\mathbf{x})$ **evidence**.

A **VAE** features **three probabilistic quantities**: the **encoder** $q_\phi(\mathbf{z} | \mathbf{x}) \approx p(\mathbf{z} | \mathbf{x})$, the **decoder** $p_\theta(\mathbf{x} | \mathbf{z}) \approx p(\mathbf{x} | \mathbf{z})$, and the **prior** $p_\lambda(\mathbf{z}) \approx p(\mathbf{z})$ of the latent variable.

VAE: Bayesian interpretation

Bayes' rule: suppose $\mathbf{z}, \mathbf{x}$ are events, p denotes probability, and $p(\mathbf{x}) \neq 0$. Then

$$p(\mathbf{z} | \mathbf{x}) = \frac{p(\mathbf{x} | \mathbf{z})p(\mathbf{z})}{p(\mathbf{x})} \Rightarrow p(\mathbf{z} | \mathbf{x}) \propto p(\mathbf{x} | \mathbf{z})p(\mathbf{z}),$$

$p(\mathbf{z} | \mathbf{x})$ **posterior probability**, $p(\mathbf{x} | \mathbf{z})$ **likelihood** of $\mathbf{z}$ given $\mathbf{x}$, $p(\mathbf{z})$ **prior probability**, $p(\mathbf{x})$ **evidence**.

A **VAE** features **three probabilistic quantities**: the **encoder** $q_\phi(\mathbf{z} | \mathbf{x}) \approx p(\mathbf{z} | \mathbf{x})$, the **decoder** $p_\theta(\mathbf{x} | \mathbf{z}) \approx p(\mathbf{x} | \mathbf{z})$, and the **prior** $p_\lambda(\mathbf{z}) \approx p(\mathbf{z})$ of the latent variable.

Fix latent space distribution, model data distribution by decoder using **marginal distribution**:

$$p_\theta(\mathbf{x}) := \int p_\theta(\mathbf{x} | \mathbf{z})p_\lambda(\mathbf{z})d\mathbf{z}, \quad \lambda \text{ fixed, } \theta \text{ variable.}$$

VAE: Bayesian interpretation

Bayes' rule: suppose $\mathbf{z}, \mathbf{x}$ are events, p denotes probability, and $p(\mathbf{x}) \neq 0$. Then

$$p(\mathbf{z} | \mathbf{x}) = \frac{p(\mathbf{x} | \mathbf{z})p(\mathbf{z})}{p(\mathbf{x})} \Rightarrow p(\mathbf{z} | \mathbf{x}) \propto p(\mathbf{x} | \mathbf{z})p(\mathbf{z}),$$

$p(\mathbf{z} | \mathbf{x})$ **posterior probability**, $p(\mathbf{x} | \mathbf{z})$ **likelihood** of $\mathbf{z}$ given $\mathbf{x}$, $p(\mathbf{z})$ **prior probability**, $p(\mathbf{x})$ **evidence**.

A **VAE** features **three probabilistic quantities**: the **encoder** $q_\phi(\mathbf{z} | \mathbf{x}) \approx p(\mathbf{z} | \mathbf{x})$, the **decoder** $p_\theta(\mathbf{x} | \mathbf{z}) \approx p(\mathbf{x} | \mathbf{z})$, and the **prior** $p_\lambda(\mathbf{z}) \approx p(\mathbf{z})$ of the latent variable.

Fix latent space distribution, model data distribution by decoder using **marginal distribution**:

$$p_\theta(\mathbf{x}) := \int p_\theta(\mathbf{x} | \mathbf{z})p_\lambda(\mathbf{z})d\mathbf{z}, \quad \lambda \text{ fixed, } \theta \text{ variable.}$$

Write **log-probability** of data distribution using the **encoder**:

$$\ln p_\theta(\mathbf{x}) = \ln \mathbb{E}_{q_\phi(\mathbf{z} | \mathbf{x})} \left[\frac{p_\theta(\mathbf{x} | \mathbf{z})p_\lambda(\mathbf{z})}{q_\phi(\mathbf{z} | \mathbf{x})} \right].$$

VAE: maximize ELBO

We want to **maximize the log-probability** of data distribution **over the parameters** ϕ (encoder) and θ (decoder) $\rightsquigarrow$ **maximum (log-)likelihood**: given $\mathbf{x}$, find “best” θ such that evidence $p_{\theta}(\mathbf{x})$ is maximal.

VAE: maximize ELBO

We want to **maximize the log-probability** of data distribution **over the parameters** ϕ (encoder) and θ (decoder) $\rightsquigarrow$ **maximum (log-)likelihood**: given $\mathbf{x}$, find “best” θ such that evidence $p_{\theta}(\mathbf{x})$ is maximal.

Jensen's inequality for probabilities: f **convex**, then $f(\mathbb{E}[X]) \leq \mathbb{E}[f(X)]$. Example: $f = -\ln$.

VAE: maximize ELBO

We want to **maximize the log-probability** of data distribution **over the parameters** ϕ (encoder) and θ (decoder) $\rightsquigarrow$ **maximum (log-)likelihood**: given $\mathbf{x}$, find “best” θ such that evidence $p_{\theta}(\mathbf{x})$ is maximal.

Jensen's inequality for probabilities: f **convex**, then $f(\mathbb{E}[X]) \leq \mathbb{E}[f(X)]$. Example: $f = -\ln$.

Evidence lower bound (ELBO) $\mathcal{L}$:

$$\begin{aligned}\ln p_{\theta}(\mathbf{x}) &= \ln \mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x})} \left[\frac{p_{\theta}(\mathbf{x} | \mathbf{z}) p_{\lambda}(\mathbf{z})}{q_{\phi}(\mathbf{z} | \mathbf{x})} \right] \geq \mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x})} \ln \left[\frac{p_{\theta}(\mathbf{x} | \mathbf{z}) p_{\lambda}(\mathbf{z})}{q_{\phi}(\mathbf{z} | \mathbf{x})} \right] \\ &= -D_{\text{KL}}(q_{\phi}(\mathbf{z} | \mathbf{x}) \| p_{\lambda}(\mathbf{z})) + \mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x})} [\ln p_{\theta}(\mathbf{x} | \mathbf{z})] =: \mathcal{L}(\mathbf{x}; \phi, \theta).\end{aligned}$$

Second part of ELBO is **negative reconstruction error**. Maximize ELBO = minimize cost of VAE.

VAE: maximize ELBO

We want to **maximize the log-probability** of data distribution **over the parameters** ϕ (encoder) and θ (decoder) $\rightsquigarrow$ **maximum (log-)likelihood**: given $\mathbf{x}$, find “best” θ such that evidence $p_{\theta}(\mathbf{x})$ is maximal.

Jensen's inequality for probabilities: f **convex**, then $f(\mathbb{E}[X]) \leq \mathbb{E}[f(X)]$. Example: $f = -\ln$.

Evidence lower bound (ELBO) $\mathcal{L}$:

$$\begin{aligned} \ln p_{\theta}(\mathbf{x}) &= \ln \mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x})} \left[\frac{p_{\theta}(\mathbf{x} | \mathbf{z}) p_{\lambda}(\mathbf{z})}{q_{\phi}(\mathbf{z} | \mathbf{x})} \right] \geq \mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x})} \ln \left[\frac{p_{\theta}(\mathbf{x} | \mathbf{z}) p_{\lambda}(\mathbf{z})}{q_{\phi}(\mathbf{z} | \mathbf{x})} \right] \\ &= -D_{\text{KL}}(q_{\phi}(\mathbf{z} | \mathbf{x}) \parallel p_{\lambda}(\mathbf{z})) + \mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x})} [\ln p_{\theta}(\mathbf{x} | \mathbf{z})] =: \mathcal{L}(\mathbf{x}; \phi, \theta). \end{aligned}$$

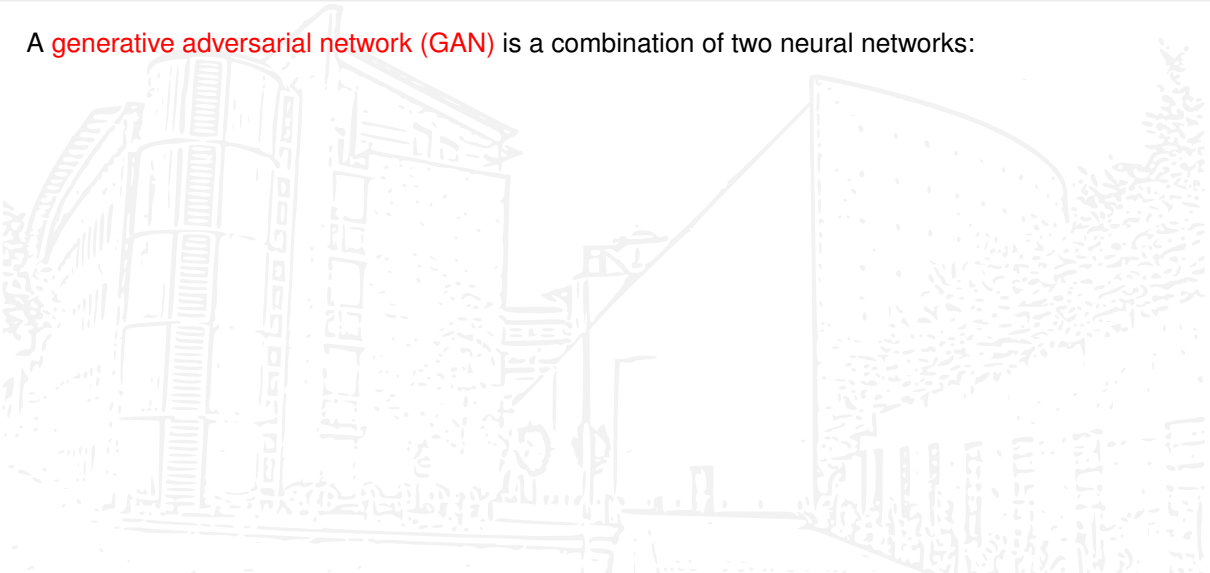
Second part of ELBO is **negative reconstruction error**. Maximize ELBO = minimize cost of VAE.

Using a **Gaussian prior** p_{λ} sometimes incurs over-regularization termed “**posterior-collapse**”:



Generative adversarial networks (GANs)

A **generative adversarial network (GAN)** is a combination of two neural networks:



Generative adversarial networks (GANs)

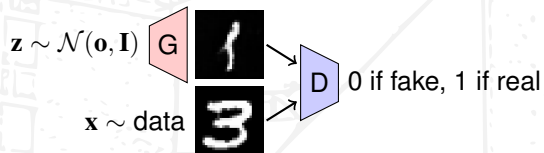
A **generative adversarial network (GAN)** is a combination of two neural networks:

- ▶ A **generative network** generates candidates that look similar to real-world examples,
- ▶ a **discriminative network** that evaluates candidates to distinguish real from generated ones.

Generative adversarial networks (GANs)

A **generative adversarial network (GAN)** is a combination of two neural networks:

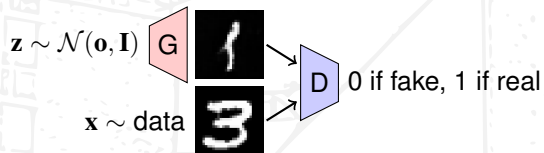
- ▶ A **generative network** generates candidates that look similar to real-world examples,
- ▶ a **discriminative network** that evaluates candidates to distinguish real from generated ones.



Generative adversarial networks (GANs)

A **generative adversarial network (GAN)** is a combination of two neural networks:

- ▶ A **generative network** generates candidates that look similar to real-world examples,
- ▶ a **discriminative network** that evaluates candidates to distinguish real from generated ones.

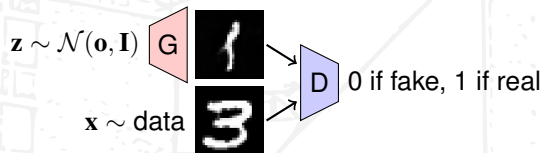


These two networks **are trained jointly** such that both of them improve until we have two very good networks that are good in their respective tasks.

Generative adversarial networks (GANs)

A **generative adversarial network (GAN)** is a combination of two neural networks:

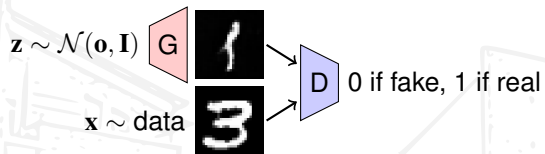
- ▶ A **generative network** generates candidates that look similar to real-world examples,
- ▶ a **discriminative network** that evaluates candidates to distinguish real from generated ones.



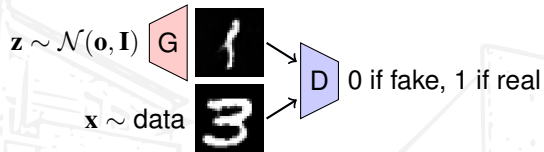
These two networks **are trained jointly** such that both of them improve until we have two very good networks that are good in their respective tasks.

GANs have been **invented in 2014** by Goodfellow et al. [4]. According to the Wikipedia entry on GANs, Yann LeCun described GANs 2016 as “**the coolest idea in machine learning in the last twenty years**”.

Training of a GAN



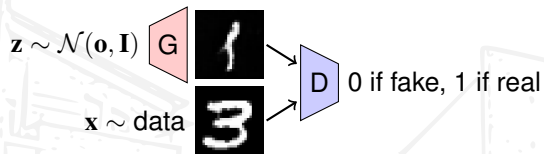
Training of a GAN



Cost function for the **discriminator** (wants to find correct labels), often w/ random noise on labels:

$$C_D := \frac{1}{N_{\text{real}}} \sum_{i=1}^{N_{\text{real}}} (\mathbf{D}(\mathbf{x}_{\text{real},i}) - 1)^2 + \frac{1}{N_{\text{fake}}} \sum_{j=1}^{N_{\text{fake}}} (\mathbf{D}(\mathbf{x}_{\text{fake},j}))^2, \quad \mathbf{x}_{\text{fake},j} := \mathbf{G}(\mathbf{z}_j)$$

Training of a GAN



Cost function for the **discriminator** (wants to find correct labels), often w/ random noise on labels:

$$C_D := \frac{1}{N_{\text{real}}} \sum_{i=1}^{N_{\text{real}}} (\mathbf{D}(\mathbf{x}_{\text{real},i}) - 1)^2 + \frac{1}{N_{\text{fake}}} \sum_{j=1}^{N_{\text{fake}}} (\mathbf{D}(\mathbf{x}_{\text{fake},j}))^2, \quad \mathbf{x}_{\text{fake},j} := \mathbf{G}(\mathbf{z}_j)$$

Cost function for the **generator** (wants to ensure wrong fake labels):

$$C_G := \frac{1}{N_{\text{fake}}} \sum_{j=1}^{N_{\text{fake}}} (\mathbf{D}(\mathbf{x}_{\text{fake},j}) - 1)^2$$

Generator training: **backprop through both**, but **update only generator** weights.

Game theory & Nash equilibrium

Training of discriminator and generator w/ binary cross-entropy loss is **two-player minimax game**:

$$\min_{\mathbf{D}} \max_{\mathbf{G}} V(\mathbf{D}, \mathbf{G}) = \min_{\mathbf{D}} \max_{\mathbf{G}} \left(\mathbb{E}_{\mathbf{x} \sim p_{\text{data}}(\mathbf{x})} [\log \mathbf{D}(\mathbf{x})] + \mathbb{E}_{\mathbf{z} \sim p_{\mathbf{z}}(\mathbf{z})} [\log(1 - \mathbf{D}(\mathbf{G}(\mathbf{z})))] \right).$$

Game theory & Nash equilibrium

Training of discriminator and generator w/ binary cross-entropy loss is **two-player minimax game**:

$$\min_{\mathbf{D}} \max_{\mathbf{G}} V(\mathbf{D}, \mathbf{G}) = \min_{\mathbf{D}} \max_{\mathbf{G}} \left(\mathbb{E}_{\mathbf{x} \sim p_{\text{data}}(\mathbf{x})} [\log \mathbf{D}(\mathbf{x})] + \mathbb{E}_{\mathbf{z} \sim p_{\mathbf{z}}(\mathbf{z})} [\log(1 - \mathbf{D}(\mathbf{G}(\mathbf{z}))) \right] .$$

Game theory:

- ▶ GAN = **two-player zero-sum non-cooperative (minimax) game**
- ▶ convergence of a GAN: **Nash equilibrium** $p_{\mathbf{G}} = p_{\text{data}}$
- ▶ In practice: discriminator needs to be trained sufficiently in each step

Game theory & Nash equilibrium

Training of discriminator and generator w/ binary cross-entropy loss is **two-player minimax game**:

$$\min_{\mathbf{D}} \max_{\mathbf{G}} V(\mathbf{D}, \mathbf{G}) = \min_{\mathbf{D}} \max_{\mathbf{G}} \left(\mathbb{E}_{\mathbf{x} \sim p_{\text{data}}(\mathbf{x})} [\log \mathbf{D}(\mathbf{x})] + \mathbb{E}_{\mathbf{z} \sim p_{\mathbf{z}}(\mathbf{z})} [\log(1 - \mathbf{D}(\mathbf{G}(\mathbf{z})))] \right).$$

Game theory:

- ▶ GAN = **two-player zero-sum non-cooperative (minimax) game**
- ▶ convergence of a GAN: **Nash equilibrium** $p_{\mathbf{G}} = p_{\text{data}}$
- ▶ In practice: discriminator needs to be trained sufficiently in each step

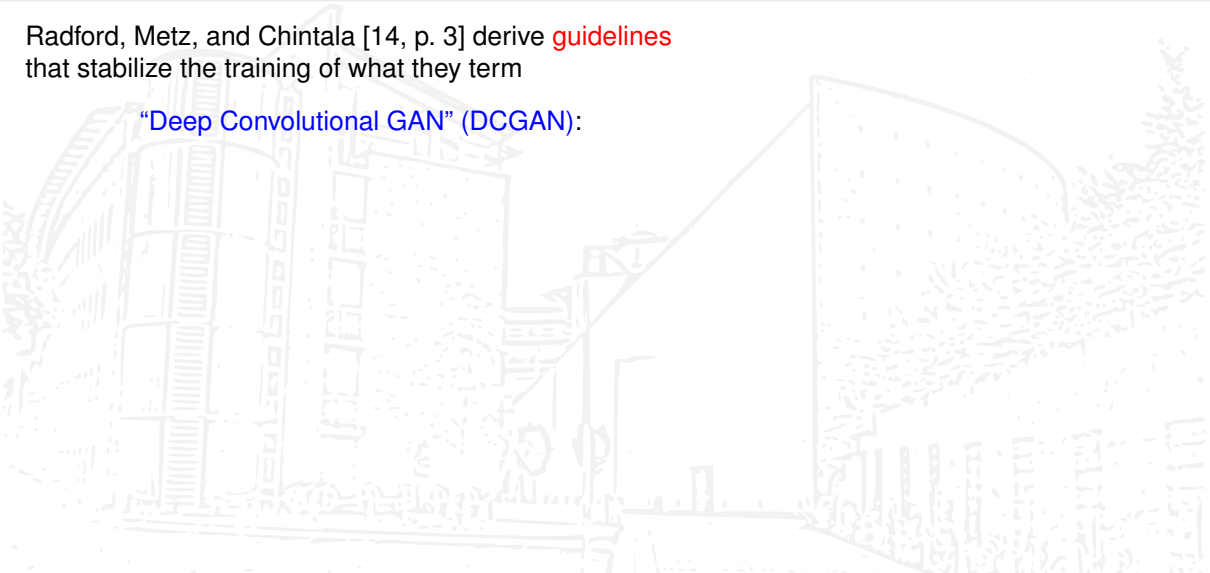
Problems:

- ▶ GAN **training unstable**, e.g., **oscillations** $\rightsquigarrow$ how to stabilize?
- ▶ **Loss functions do not decay** to zero. **When to stop?**
- ▶ How to measure **success**? Want to minimize distance between distributions of training data and of generated examples $\rightsquigarrow$ **Wasserstein distances**.

Deep convolutional GAN (DCGAN)

Radford, Metz, and Chintala [14, p. 3] derive **guidelines** that stabilize the training of what they term

“Deep Convolutional GAN” (DCGAN):



Deep convolutional GAN (DCGAN)

Radford, Metz, and Chintala [14, p. 3] derive **guidelines** that stabilize the training of what they term

“Deep Convolutional GAN” (DCGAN):

- ▶ **Replace** any **pooling** layers with **strided convolutions** (discriminator) and **fractional-strided convolutions** (generator).
- ▶ Use **batchnorm** in **both** the generator and the discriminator.
- ▶ **Remove fully connected hidden layers** for deeper architectures.
- ▶ Use **ReLU** activation in **generator** for all layers except for the **output**, which uses **TanH**.
- ▶ Use **leaky ReLU** activation in the **discriminator** for all layers.

Deep convolutional GAN (DCGAN)

Radford, Metz, and Chintala [14, p. 3] derive **guidelines** that stabilize the training of what they term

“Deep Convolutional GAN” (DCGAN):

- ▶ Replace any **pooling** layers with **strided convolutions** (discriminator) and **fractional-strided convolutions** (generator).
- ▶ Use **batchnorm** in **both** the generator and the discriminator.
- ▶ Remove **fully connected hidden layers** for deeper architectures.
- ▶ Use **ReLU** activation in **generator** for all layers except for the **output**, which uses **TanH**.
- ▶ Use **leaky ReLU** activation in the **discriminator** for all layers.

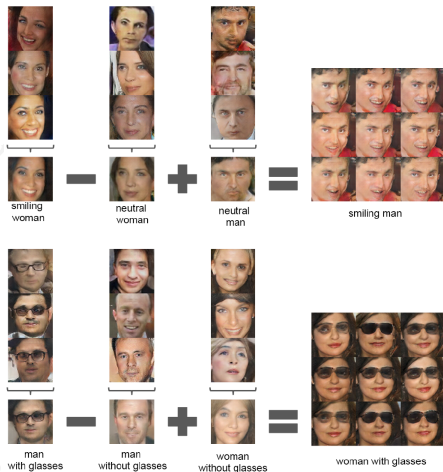


Figure taken from Radford, Metz, and Chintala [14].

Joint probabilities, marginal probabilities $\rightsquigarrow$ transport plans

Joint probability distribution $p(x, y)$ has marginal distributions

$$p(x) = \int p(x, y) dy, \quad p(y) = \int p(x, y) dx$$

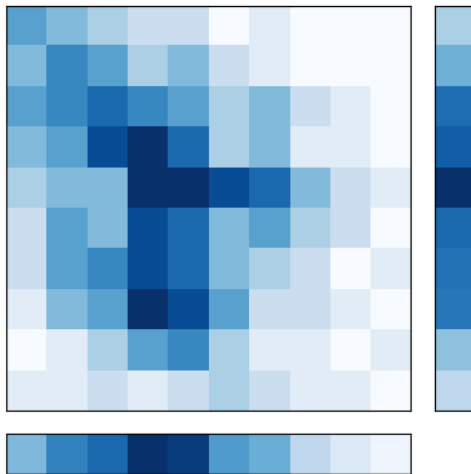
Joint probabilities, marginal probabilities $\rightsquigarrow$ transport plans

Joint probability distribution $p(x, y)$ has marginal distributions

$$p(x) = \int p(x, y) dy, \quad p(y) = \int p(x, y) dx$$

Discrete case: matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$ w/ $a_{i,j} \geq 0$,

$$\sum_{i,j} a_{i,j} = 1, \quad \boldsymbol{\mu} = \mathbf{A} \mathbf{e}, \quad \boldsymbol{\nu}^\top = \mathbf{e}^\top \mathbf{A}.$$



Joint probabilities, marginal probabilities $\rightsquigarrow$ transport plans

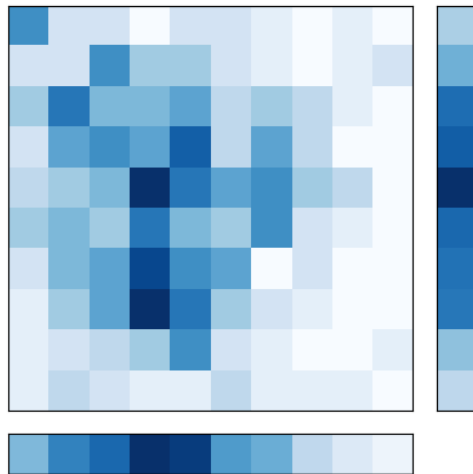
Joint probability distribution $p(x, y)$ has marginal distributions

$$p(x) = \int p(x, y) dy, \quad p(y) = \int p(x, y) dx$$

Discrete case: matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$ w/ $a_{i,j} \geq 0$,

$$\sum_{i,j} a_{i,j} = 1, \quad \boldsymbol{\mu} = \mathbf{A} \mathbf{e}, \quad \boldsymbol{\nu}^\top = \mathbf{e}^\top \mathbf{A}.$$

Given $\boldsymbol{\mu}, \boldsymbol{\nu}$, construct $\mathbf{A}$: non-unique.



Joint probabilities, marginal probabilities $\rightsquigarrow$ transport plans

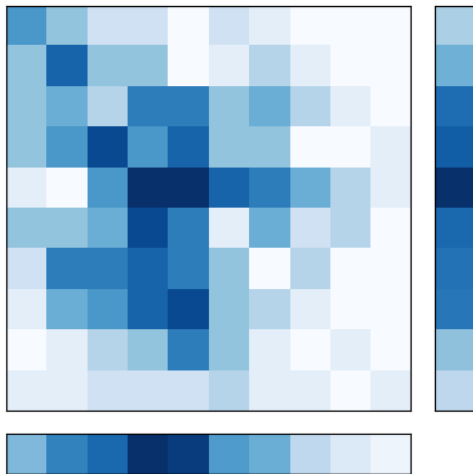
Joint probability distribution $p(x, y)$ has marginal distributions

$$p(x) = \int p(x, y) dy, \quad p(y) = \int p(x, y) dx$$

Discrete case: matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$ w/ $a_{i,j} \geq 0$,

$$\sum_{i,j} a_{i,j} = 1, \quad \boldsymbol{\mu} = \mathbf{A} \mathbf{e}, \quad \boldsymbol{\nu}^\top = \mathbf{e}^\top \mathbf{A}.$$

Given $\boldsymbol{\mu}, \boldsymbol{\nu}$, construct $\mathbf{A}$: non-unique.



Joint probabilities, marginal probabilities $\rightsquigarrow$ transport plans

Joint probability distribution $p(x, y)$ has marginal distributions

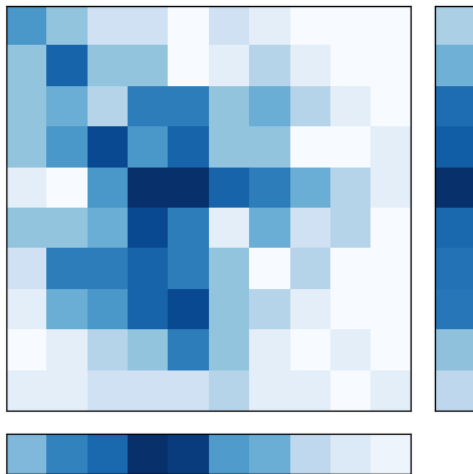
$$p(x) = \int p(x, y) dy, \quad p(y) = \int p(x, y) dx$$

Discrete case: matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$ w/ $a_{i,j} \geq 0$,

$$\sum_{i,j} a_{i,j} = 1, \quad \boldsymbol{\mu} = \mathbf{A} \mathbf{e}, \quad \boldsymbol{\nu}^\top = \mathbf{e}^\top \mathbf{A}.$$

Given $\boldsymbol{\mu}, \boldsymbol{\nu}$, construct $\mathbf{A}$: non-unique.

Interpretation of $\mathbf{A}$: transport plan



Joint probabilities, marginal probabilities $\rightsquigarrow$ transport plans

Joint probability distribution $p(x, y)$ has marginal distributions

$$p(x) = \int p(x, y) dy, \quad p(y) = \int p(x, y) dx$$

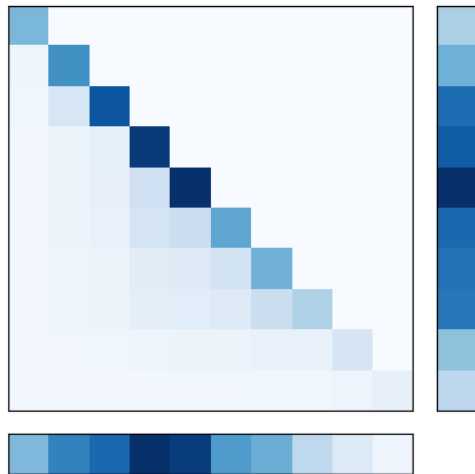
Discrete case: matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$ w/ $a_{i,j} \geq 0$,

$$\sum_{i,j} a_{i,j} = 1, \quad \boldsymbol{\mu} = \mathbf{A} \mathbf{e}, \quad \boldsymbol{\nu}^\top = \mathbf{e}^\top \mathbf{A}.$$

Given $\boldsymbol{\mu}, \boldsymbol{\nu}$, construct $\mathbf{A}$: non-unique.

Interpretation of $\mathbf{A}$: transport plan

Earth movers distance: cost to move earth from i to j proportional to $|i - j| \rightsquigarrow$ Wasserstein-1.



Wasserstein distance

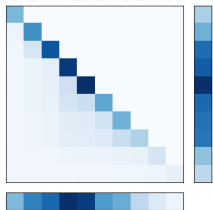
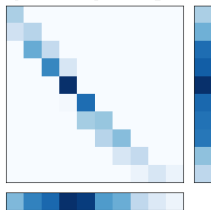
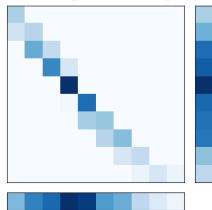
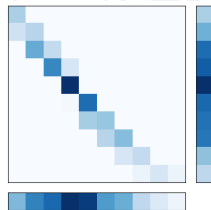
One can prove **uniqueness** for **Wasserstein- p distance**:

$$W_p(\mu, \nu) := \left(\inf_{\gamma \in \Gamma(\mu, \nu)} \iint \|\mathbf{x} - \mathbf{y}\|_2^p \gamma(\mathbf{x}, \mathbf{y}) d\mathbf{x} d\mathbf{y} \right)^{1/p}, \quad \Gamma \text{ set of prob. m. w/ marginals } \mu, \nu.$$

Wasserstein distance

One can prove **uniqueness** for **Wasserstein- p** distance:

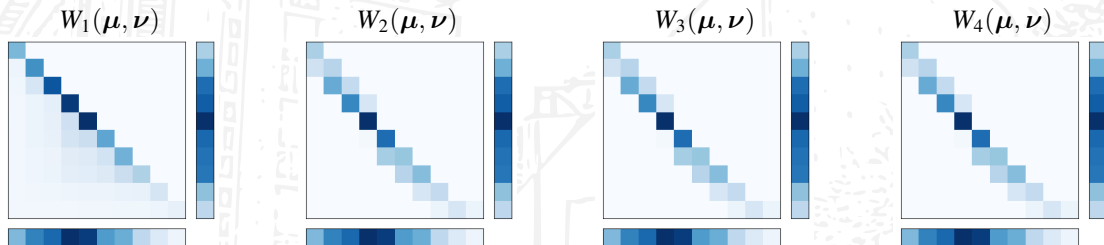
$$W_p(\mu, \nu) := \left(\inf_{\gamma \in \Gamma(\mu, \nu)} \iint \|\mathbf{x} - \mathbf{y}\|_2^p \gamma(\mathbf{x}, \mathbf{y}) d\mathbf{x} d\mathbf{y} \right)^{1/p}, \quad \Gamma \text{ set of prob. m. w/ marginals } \mu, \nu.$$

 $W_1(\mu, \nu)$  $W_2(\mu, \nu)$  $W_3(\mu, \nu)$  $W_4(\mu, \nu)$ 

Wasserstein distance

One can prove **uniqueness** for **Wasserstein- p distance**:

$$W_p(\mu, \nu) := \left(\inf_{\gamma \in \Gamma(\mu, \nu)} \iint \|\mathbf{x} - \mathbf{y}\|_2^p \gamma(\mathbf{x}, \mathbf{y}) d\mathbf{x} d\mathbf{y} \right)^{1/p}, \quad \Gamma \text{ set of prob. m. w/ marginals } \mu, \nu.$$



Wasserstein-2 distance for **Gaussian measures** (compare w/ **Kullback-Leibler divergence**):

$$W_2(\mathcal{N}_0, \mathcal{N}_1)^2 = \|\mu_1 - \mu_0\|_2^2 + \text{trace}(\Sigma_0 + \Sigma_1 - 2(\Sigma_0^{1/2} \Sigma_1 \Sigma_0^{1/2})^{1/2}).$$

Linear programming, Kantorovich-Rubinstein duality

How to find **Wasserstein-1 distance**, e.g., how to **minimize**? Consider

$$W_1(\boldsymbol{\mu}, \boldsymbol{\nu}) = \min_{\mathbf{A}} \sum_{i,j} a_{i,j} |i - j| \quad \text{s.t.} \quad \mathbf{A}\mathbf{e} = \boldsymbol{\mu}, \quad \mathbf{e}^T \mathbf{A} = \boldsymbol{\nu}^T, \quad a_{i,j} \geq 0.$$

Linear programming, Kantorovich-Rubinstein duality

How to find **Wasserstein-1 distance**, e.g., how to **minimize**? Consider

$$W_1(\boldsymbol{\mu}, \boldsymbol{\nu}) = \min_{\mathbf{A}} \sum_{i,j} a_{i,j} |i - j| \quad \text{s.t.} \quad \mathbf{A}\mathbf{e} = \boldsymbol{\mu}, \quad \mathbf{e}^T \mathbf{A} = \boldsymbol{\nu}^T, \quad a_{i,j} \geq 0.$$

The **vec-operator** stacks columns into long column vector, w/ **Kronecker product** we have

$$\text{vec}(\mathbf{A}\mathbf{X}\mathbf{B}) = (\mathbf{B}^T \otimes \mathbf{A}) \text{vec}(\mathbf{X}), \quad \text{vec}(\mathbf{x}^T) = \text{vec}(\mathbf{x}) = \mathbf{x}.$$

Linear programming, Kantorovich-Rubinstein duality

How to find **Wasserstein-1 distance**, e.g., how to **minimize**? Consider

$$W_1(\boldsymbol{\mu}, \boldsymbol{\nu}) = \min_{\mathbf{A}} \sum_{i,j} a_{i,j} |i - j| \quad \text{s.t.} \quad \mathbf{A}\mathbf{e} = \boldsymbol{\mu}, \quad \mathbf{e}^T \mathbf{A} = \boldsymbol{\nu}^T, \quad a_{i,j} \geq 0.$$

The **vec-operator** stacks columns into long column vector, w/ **Kronecker product** we have

$$\text{vec}(\mathbf{A}\mathbf{X}\mathbf{B}) = (\mathbf{B}^T \otimes \mathbf{A}) \text{vec}(\mathbf{X}), \quad \text{vec}(\mathbf{x}^T) = \text{vec}(\mathbf{x}) = \mathbf{x}.$$

Write W_1 as **linear program** ($\mathbf{D}$ defined by elements $d_{i,j} = |i - j|$):

$$\min_{\mathbf{A}} \text{vec}(\mathbf{D})^T \text{vec}(\mathbf{A}) \quad \text{s.t.} \quad \begin{pmatrix} \mathbf{e}^T \otimes \mathbf{I} \\ \mathbf{I} \otimes \mathbf{e}^T \end{pmatrix} \text{vec}(\mathbf{A}) = \begin{pmatrix} \boldsymbol{\mu} \\ \boldsymbol{\nu} \end{pmatrix} =: \mathbf{b}, \quad \text{vec}(\mathbf{A}) \geq \mathbf{o}.$$

Linear programming, Kantorovich-Rubinstein duality

How to find **Wasserstein-1 distance**, e.g., how to **minimize**? Consider

$$W_1(\mu, \nu) = \min_{\mathbf{A}} \sum_{i,j} a_{i,j} |i - j| \quad \text{s.t.} \quad \mathbf{A}\mathbf{e} = \mu, \quad \mathbf{e}^\top \mathbf{A} = \nu^\top, \quad a_{i,j} \geq 0.$$

The **vec-operator** stacks columns into long column vector, w/ **Kronecker product** we have

$$\text{vec}(\mathbf{AXB}) = (\mathbf{B}^\top \otimes \mathbf{A}) \text{vec}(\mathbf{X}), \quad \text{vec}(\mathbf{x}^\top) = \text{vec}(\mathbf{x}) = \mathbf{x}.$$

Write W_1 as **linear program** ($\mathbf{D}$ defined by elements $d_{i,j} = |i - j|$):

$$\min_{\mathbf{A}} \text{vec}(\mathbf{D})^\top \text{vec}(\mathbf{A}) \quad \text{s.t.} \quad \begin{pmatrix} \mathbf{e}^\top \otimes \mathbf{I} \\ \mathbf{I} \otimes \mathbf{e}^\top \end{pmatrix} \text{vec}(\mathbf{A}) = \begin{pmatrix} \mu \\ \nu \end{pmatrix} =: \mathbf{b}, \quad \text{vec}(\mathbf{A}) \geq \mathbf{o}.$$

Dual formulation w/ dual variable $\mathbf{y}^\top = (\mathbf{f}^\top, -\mathbf{g}^\top)$:

$$\max_{\mathbf{y}} \mathbf{b}^\top \mathbf{y} = \mu^\top \mathbf{f} - \nu^\top \mathbf{g} \quad \text{s.t.} \quad \begin{pmatrix} \mathbf{e}^\top \otimes \mathbf{I} \\ \mathbf{I} \otimes \mathbf{e}^\top \end{pmatrix}^\top \begin{pmatrix} \mathbf{f} \\ -\mathbf{g} \end{pmatrix} \leq \text{vec}(\mathbf{D}) \quad \Leftrightarrow \quad \mathbf{f}\mathbf{e}^\top - \mathbf{e}\mathbf{g}^\top \leq \mathbf{D}.$$

Dual form of Wasserstein-1

Want to **maximize**

$$\mathbf{b}^\top \mathbf{y} = \sum_{i=1}^n f_i \mu_i - \sum_{i=1}^n g_i \nu_i = \mathbb{E}_{i \sim \mu}[f(i)] - \mathbb{E}_{i \sim \nu}[g(i)] \quad \text{s.t.} \quad \mathbf{C}(\mathbf{f}, \mathbf{g}) := \mathbf{f} \mathbf{e}^\top - \mathbf{e} \mathbf{g}^\top \leq \mathbf{D}.$$

Dual form of Wasserstein-1

Want to **maximize**

$$\mathbf{b}^\top \mathbf{y} = \sum_{i=1}^n f_i \mu_i - \sum_{i=1}^n g_i \nu_i = \mathbb{E}_{i \sim \mu}[f(i)] - \mathbb{E}_{i \sim \nu}[g(i)] \quad \text{s.t.} \quad \mathbf{C}(\mathbf{f}, \mathbf{g}) := \mathbf{f} \mathbf{e}^\top - \mathbf{e} \mathbf{g}^\top \leq \mathbf{D}.$$

Need **large $\mathbf{f}$** and **small $\mathbf{g}$** . Looking at elements $c_{i,j}(\mathbf{f}, \mathbf{g}) = f_i - g_j \leq |i - j|$, $1 \leq i, j \leq n$ gives

$$c_{i,i}(\mathbf{f}, \mathbf{g}) = f_i - g_i \leq |i - i| = 0 \quad \Rightarrow \quad \mathbf{f} \leq \mathbf{g},$$

$$c_{i,j}(\mathbf{f}, \mathbf{g}) + c_{j,i}(\mathbf{f}, \mathbf{g}) = (f_i - g_j) + (f_j - g_i) = (f_i - g_i) + (f_j - g_j) \leq 0 \quad \Rightarrow \quad \mathbf{C}(\mathbf{f}, \mathbf{g}) + \mathbf{C}(\mathbf{f}, \mathbf{g})^\top \leq \mathbf{O}.$$

Dual form of Wasserstein-1

Want to **maximize**

$$\mathbf{b}^\top \mathbf{y} = \sum_{i=1}^n f_i \mu_i - \sum_{i=1}^n g_i \nu_i = \mathbb{E}_{i \sim \mu}[f(i)] - \mathbb{E}_{i \sim \nu}[g(i)] \quad \text{s.t.} \quad \mathbf{C}(\mathbf{f}, \mathbf{g}) := \mathbf{f} \mathbf{e}^\top - \mathbf{e} \mathbf{g}^\top \leq \mathbf{D}.$$

Need **large f** and **small g**. Looking at elements $c_{i,j}(\mathbf{f}, \mathbf{g}) = f_i - g_j \leq |i - j|$, $1 \leq i, j \leq n$ gives

$$c_{i,i}(\mathbf{f}, \mathbf{g}) = f_i - g_i \leq |i - i| = 0 \quad \Rightarrow \quad \mathbf{f} \leq \mathbf{g},$$

$$c_{i,j}(\mathbf{f}, \mathbf{g}) + c_{j,i}(\mathbf{f}, \mathbf{g}) = (f_i - g_j) + (f_j - g_i) = (f_i - g_i) + (f_j - g_j) \leq 0 \quad \Rightarrow \quad \mathbf{C}(\mathbf{f}, \mathbf{g}) + \mathbf{C}(\mathbf{f}, \mathbf{g})^\top \leq \mathbf{O}.$$

Unique solution (thus $W_1(\mu, \nu) = \max \mathbf{b}^\top \mathbf{y}$ using Farkas theorem):

$$\mathbf{g} = \mathbf{f}, \quad |f_i - f_j| \leq |i - j| \quad \Rightarrow \quad f_i = f(i) \text{ Lipschitz w/ constant } L = 1.$$

Dual form of Wasserstein-1

Want to **maximize**

$$\mathbf{b}^\top \mathbf{y} = \sum_{i=1}^n f_i \mu_i - \sum_{i=1}^n g_i \nu_i = \mathbb{E}_{i \sim \mu}[f(i)] - \mathbb{E}_{i \sim \nu}[g(i)] \quad \text{s.t.} \quad \mathbf{C}(\mathbf{f}, \mathbf{g}) := \mathbf{f} \mathbf{e}^\top - \mathbf{e} \mathbf{g}^\top \leq \mathbf{D}.$$

Need **large f** and **small g**. Looking at elements $c_{i,j}(\mathbf{f}, \mathbf{g}) = f_i - g_j \leq |i - j|$, $1 \leq i, j \leq n$ gives

$$c_{i,i}(\mathbf{f}, \mathbf{g}) = f_i - g_i \leq |i - i| = 0 \quad \Rightarrow \quad \mathbf{f} \leq \mathbf{g},$$

$$c_{i,j}(\mathbf{f}, \mathbf{g}) + c_{j,i}(\mathbf{f}, \mathbf{g}) = (f_i - g_j) + (f_j - g_i) = (f_i - g_i) + (f_j - g_j) \leq 0 \quad \Rightarrow \quad \mathbf{C}(\mathbf{f}, \mathbf{g}) + \mathbf{C}(\mathbf{f}, \mathbf{g})^\top \leq \mathbf{O}.$$

Unique solution (thus $W_1(\mu, \nu) = \max \mathbf{b}^\top \mathbf{y}$ using Farkas theorem):

$$\mathbf{g} = \mathbf{f}, \quad |f_i - f_j| \leq |i - j| \quad \Rightarrow \quad f_i = f(i) \quad \text{Lipschitz w/ constant } L = 1.$$

Continuous version ($\Gamma(\mu, \nu)$ set of all joint probabilities w/ marginals μ, ν):

$$W_1(\mu, \nu) = \inf_{\gamma \in \Gamma(\mu, \nu)} \iint \|\mathbf{x} - \mathbf{y}\| \gamma(\mathbf{x}, \mathbf{y}) d\mathbf{x} d\mathbf{y} = \sup_{\|f\|_L \leq 1} \mathbb{E}_{t \sim \mu}[f(t)] - \mathbb{E}_{t \sim \nu}[f(t)].$$

Wasserstein GAN (WGAN)

Observations on **Lipschitz continuity**:

- ▶ A **locally Lipschitz** function on **compactum** is Lipschitz, scaling does only scale value of W_1 .
- ▶ **Differentiable** functions are Lipschitz, constant determined by **norm of gradient**.

Wasserstein GAN (WGAN)

Observations on **Lipschitz continuity**:

- ▶ A **locally Lipschitz** function on **compactum** is Lipschitz, scaling does only scale value of W_1 .
- ▶ **Differentiable** functions are Lipschitz, constant determined by **norm of gradient**.

Wasserstein GAN (WGAN): discriminator $\rightsquigarrow$
“critic” measures W_1 by **“clamping”** the weights, critic trained to optimality,
 Arjovsky, Chintala, and Bottou [2]

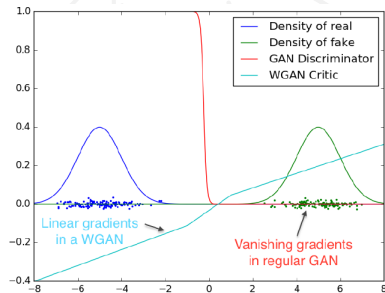


Figure 2: Optimal discriminator and critic when learning to differentiate two Gaussians. As we can see, the discriminator of a minimax GAN saturates and results in vanishing gradients. Our WGAN critic provides very clean gradients on all parts of the space.

Figure taken from Arjovsky, Chintala, and Bottou [2].

Wasserstein GAN (WGAN)

Observations on **Lipschitz continuity**:

- ▶ A **locally Lipschitz** function on **compactum** is Lipschitz, scaling does only scale value of W_1 .
- ▶ **Differentiable** functions are Lipschitz, constant determined by **norm of gradient**.

Wasserstein GAN (WGAN): discriminator $\rightsquigarrow$ “**critic**” measures W_1 by “**clamping**” the weights, critic trained to optimality, Arjovsky, Chintala, and Bottou [2]

WGAN w/ gradient penalizing (WGAN-GP): **penalize norm** of gradient of critic to be close to 1, Gulrajani et al. [5]

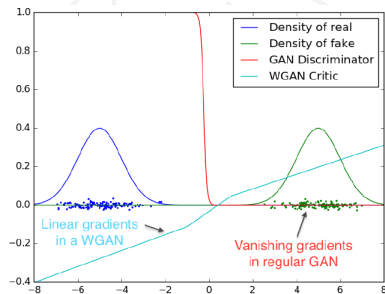


Figure 2: Optimal discriminator and critic when learning to differentiate two Gaussians. As we can see, the discriminator of a minimax GAN saturates and results in vanishing gradients. Our WGAN critic provides very clean gradients on all parts of the space.

Figure taken from Arjovsky, Chintala, and Bottou [2].

Wasserstein GAN (WGAN)

Observations on **Lipschitz continuity**:

- ▶ A **locally Lipschitz** function on **compactum** is Lipschitz, scaling does only scale value of W_1 .
- ▶ **Differentiable** functions are Lipschitz, constant determined by **norm of gradient**.

Wasserstein GAN (WGAN): discriminator $\rightsquigarrow$ “**critic**” measures W_1 by “**clamping**” the weights, critic trained to optimality, Arjovsky, Chintala, and Bottou [2]

WGAN w/ gradient penalizing (WGAN-GP): **penalize norm** of gradient of critic to be close to 1, Gulrajani et al. [5]

Banach WGAN (BWGAN): extends WGAN-GP to **Banach spaces**, Adler and Lunz [1]

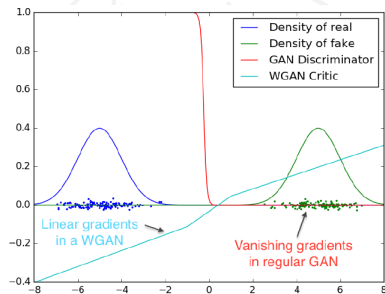


Figure 2: Optimal discriminator and critic when learning to differentiate two Gaussians. As we can see, the discriminator of a minimax GAN saturates and results in vanishing gradients. Our WGAN critic provides very clean gradients on all parts of the space.

Figure taken from Arjovsky, Chintala, and Bottou [2].

Wasserstein GAN (WGAN)

Observations on **Lipschitz continuity**:

- ▶ A **locally Lipschitz** function on **compactum** is Lipschitz, scaling does only scale value of W_1 .
- ▶ **Differentiable** functions are Lipschitz, constant determined by **norm of gradient**.

Wasserstein GAN (WGAN): discriminator $\rightsquigarrow$ “critic” measures W_1 by “clamping” the weights, critic trained to optimality, Arjovsky, Chintala, and Bottou [2]

WGAN w/ gradient penalizing (WGAN-GP): penalize norm of gradient of critic to be close to 1, Gulrajani et al. [5]

Banach WGAN (BWGAN): extends WGAN-GP to **Banach spaces**, Adler and Lunz [1]

Wasserstein-2 GN: use **Wasserstein-2** distance in generative networks, Korotin et al. [11]

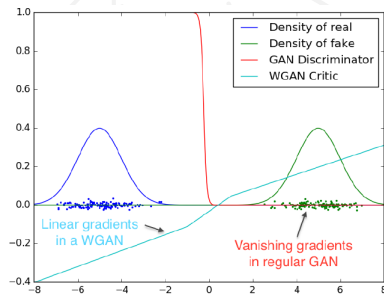


Figure 2: Optimal discriminator and critic when learning to differentiate two Gaussians. As we can see, the discriminator of a minimax GAN saturates and results in vanishing gradients. Our WGAN critic provides very clean gradients on all parts of the space.

Figure taken from Arjovsky, Chintala, and Bottou [2].

StyleGAN

A famous GAN is **StyleGAN** by Karras, Laine, and Aila [10], researchers at **NVIDIA**.

StyleGAN computes **photorealistic pictures** of people that do not exist, based on different styles.

The picture on the right shows three such **styles** for five **different types of person**.

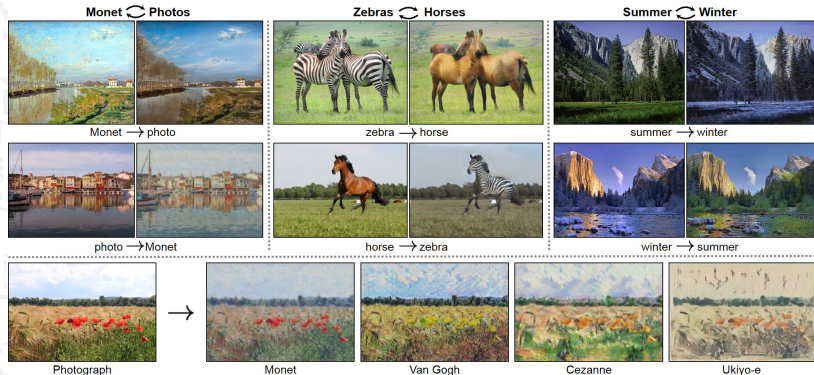


- ▶ <https://en.wikipedia.org/wiki/StyleGAN>,
- ▶ <https://github.com/NVlabs/stylegan2>,
- ▶ <https://nvlabsgithub.io/stylegan3/>
- ▶ <https://www.thispersondoesnotexist.com/>,

CycleGAN

A very funny GAN
is **CycleGAN**
by Zhu et al. [20].

“Software that can
generate photos
from paintings, turn
horses into zebras,
perform style
transfer, and more.”



CycleGAN can be found at <https://github.com/junyanz/CycleGAN>.

References I

- [1] Jonas Adler and Sebastian Lunz. “Banach Wasserstein GAN”. In: *Advances in Neural Information Processing Systems*. Ed. by S. Bengio et al. Vol. 31. Curran Associates, Inc., 2018. URL: <https://proceedings.neurips.cc/paper/2018/file/91d0dbfd38d950cb716c4dd26c5da08a-Paper.pdf>.
- [2] Martin Arjovsky, Soumith Chintala, and Léon Bottou. “Wasserstein GAN”. In: *arXiv e-prints*, arXiv:1701.07875 (Jan. 2017). arXiv: 1701.07875 [stat.ML].
- [3] H. Bouillard and Y. Kamp. “Auto-association by multilayer perceptrons and singular value decomposition”. In: *Biol. Cybernet.* 59.4-5 (1988), pp. 291–294. ISSN: 0340-1200. DOI: 10.1007/BF00332918.
- [4] Ian Goodfellow et al. “Generative Adversarial Nets”. In: *Proceedings of the International Conference on Neural Information Processing Systems (NIPS 2014)* (2014), pp. 2672–2680. URL: <https://papers.nips.cc/paper/5423-generative-adversarial-nets.pdf>.

References II

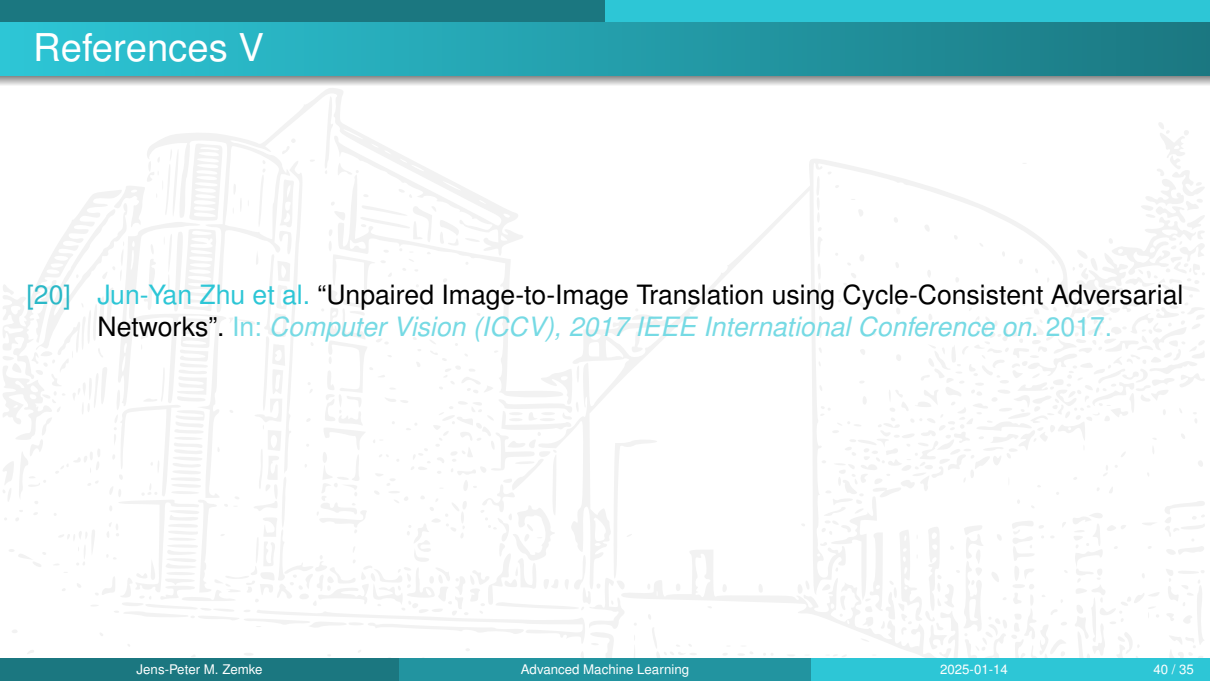
- [5] Ishaan Gulrajani et al. “Improved Training of Wasserstein GANs”. In: *Advances in Neural Information Processing Systems*. Ed. by I. Guyon et al. Vol. 30. Curran Associates, Inc., 2017. URL: <https://proceedings.neurips.cc/paper/2017/file/892c3b1c6dccd52936e27cbd0ff683d6-Paper.pdf>.
- [6] Kaiming He et al. “Deep Residual Learning for Image Recognition”. In: *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, 2016. DOI: 10.1109/CVPR.2016.90.
- [7] Kaiming He et al. “Identity Mappings in Deep Residual Networks”. In: *arXiv e-prints*, arXiv:1603.05027 (Mar. 2016). arXiv: 1603.05027 [cs.CV].
- [8] Geoffrey E Hinton and Sam Roweis. “Stochastic Neighbor Embedding”. In: *Advances in Neural Information Processing Systems*. Ed. by S. Becker, S. Thrun, and K. Obermayer. Vol. 15. MIT Press, 2003. URL: <https://proceedings.neurips.cc/paper/2002/file/6150ccc6069bea6b5716254057a194ef-Paper.pdf>.
- [9] Gao Huang et al. “Densely Connected Convolutional Networks”. In: *arXiv e-prints*, arXiv:1608.06993 (Aug. 2016). arXiv: 1608.06993 [cs.CV].

References III

- [10] Tero Karras, Samuli Laine, and Timo Aila. “A Style-Based Generator Architecture for Generative Adversarial Networks”. In: *arXiv e-prints*, arXiv:1812.04948 (Dec. 2018). arXiv: 1812.04948 [cs.NE].
- [11] Alexander Korotin et al. “Wasserstein-2 Generative Networks”. In: *International Conference on Learning Representations*. 2021. URL: https://openreview.net/forum?id=bEoxzW_EXsa.
- [12] L.J.P. van der Maaten and G.E. Hinton. “Visualizing High-Dimensional Data Using t-SNE”. In: *Journal of Machine Learning Research* 9 (Nov. 2008), pp. 2579–2605. URL: https://lvdmaaten.github.io/publications/papers/JMLR_2008.pdf.
- [13] Leland McInnes, John Healy, and James Melville. “UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction”. In: *arXiv e-prints*, arXiv:1802.03426 (Feb. 2018). arXiv: 1802.03426 [stat.ML].
- [14] Alec Radford, Luke Metz, and Soumith Chintala. “Unsupervised Representation Learning with Deep Convolutional Generative Adversarial Networks”. In: *arXiv e-prints*, arXiv:1511.06434 (Nov. 2015). arXiv: 1511.06434 [cs.LG].

References IV

- [15] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. “U-Net: Convolutional Networks for Biomedical Image Segmentation”. In: *Medical Image Computing and Computer-Assisted Intervention – MICCAI 2015*. Ed. by Nassir Navab et al. Cham: Springer International Publishing, 2015, pp. 234–241. ISBN: 978-3-319-24574-4. DOI: 10.1007/978-3-319-24574-4_28.
- [16] Rupesh Kumar Srivastava, Klaus Greff, and Jürgen Schmidhuber. “Highway Networks”. In: *arXiv e-prints*, arXiv:1505.00387 (May 2015). arXiv: 1505.00387 [cs.LG].
- [17] G. W. Stewart and Ji Guang Sun. *Matrix perturbation theory*. Computer Science and Scientific Computing. Academic Press, Inc., Boston, MA, 1990, pp. xvi+365. ISBN: 0-12-670230-6.
- [18] Christian Szegedy et al. “Going deeper with convolutions”. In: *2015 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, 2015. DOI: 10.1109/CVPR.2015.7298594.
- [19] Saining Xie et al. “Aggregated Residual Transformations for Deep Neural Networks”. In: *arXiv e-prints*, arXiv:1611.05431 (Nov. 2016). arXiv: 1611.05431 [cs.CV].

- 
- [20] Jun-Yan Zhu et al. “Unpaired Image-to-Image Translation using Cycle-Consistent Adversarial Networks”. In: *Computer Vision (ICCV), 2017 IEEE International Conference on*. 2017.